Constitutive Authorization at the Decoding Boundary: Grammar-Constrained Decoding as a Positive, Generation-Time Security Control for LLM Agents

Vince Ovando*

Draft — June 25, 2026

Abstract

Tool-call authorization for large-language-model agents is enforced at the decoding boundary, applying complete mediation [1] to the token sampler: a per-request authorization grammar G_s makes out-of-scope tokens ungenerable at the sampler [18, 19] before any token is committed. The thesis is constitutive versus corrective enforcement. A grammar-constrained decoder (`c`) and a post-parse allowlist gate (`d`) enforce the same policy over $L(G_s)$ but differ in completeness. The empirical signature is emission, the count of times the unauthorized artifact is produced: 0 under `c` by construction, > 0 under `d`. Against a single deployed agent (Tantalus), we replay capable authorization-class attacks across a multi-condition design, scoring bypass by effect (an attacker marker reaching a non-owned `fetchURL` sink) with Clopper–Pearson intervals, and add a soundness core that turns an observed 0 into a provable 0 (emitted $\in L(G_s) \subseteq$ Authorized(s)) across ∼868,000 GCD attack trials. The evaluation spans five datasets and three pretraining lineages, totaling ∼6.1 million individual inference calls and over half a billion (∼544 million) generated tokens. The capable Mistral-Small-4-119B anchor still leaks under the full behavioral stack `a4` (bypass 1.82% [1.43, 2.29], CI-lower > 0) while `c` is 0.00% [0, 0.09]; the Gemma-4-31B lineage leaks under `a4` (33.80% [31.73, 35.92]) while `c` is 0.00% [0, 0.18]; and an abliterated steelman [21] moves 97.85% → 0%. Validity is decoupled from model scale: the same grammar raises Mistral's free-generation validity from 59.2% to 99.8%. However, the guarantee is scoped to action integrity over enumerable-action sinks (the `fetchURL`, channel, path, and recipient parameters), not the confidentiality of free-text content, and the full set of limitations is detailed in the paper.

1 Introduction

LLM agents are now deployed with real tool access: they read files, query inboxes and chat logs, and issue outbound HTTP requests on behalf of a user. Most of these tool calls happen in backend pipelines the user never sees. An agent invoked to "run the daily sync" silently reads, retrieves, and forwards on the user's behalf, and only a summary returns to the screen. This creates a confused-deputy exposure that the literature has named *indirect prompt injection*: malicious instructions placed in a trusted data or skill channel (an email, a support ticket, a tool's own documentation) are ingested as content and then followed as commands, redirecting the agent's authorized capabilities

*Correspondence: `mrovandoninja@gmail.com`. Draft manuscript; affiliation omitted pending submission.

toward an attacker's ends [7]. The injection rides in through exactly the channels the agent must read to do its job.

The defenses the industry reaches for are *behavioral*: a defensive system prompt (an RFC-2119 security policy), an input classifier on the user message, an output filter scanning tool-call parameters against a credential denylist. These are negative, default-allow controls. They enumerate the bad and permit everything else, and they are permanently bypassable for a structural reason rather than a tuning one. A defensive prompt and an input classifier are *in-band*: they live in the same input channel the injection occupies, so an injection that can address the model can address them too, and overrides them. An output filter is a negative denylist, so business data carrying no credential signature sails straight through it. No amount of prompt engineering or classifier retraining removes a weakness that is a property of *where* and *how* the control is applied.

This is the same shape as the long-settled host-security shift from signature-based antivirus (a negative control that blocks known-bad) to application allowlisting (a positive control that permits only known-good). The durable lesson there was that security should be a property of *complete mediation* and *fail-safe defaults*, mediating every access and denying by default, rather than of an ever-growing catalogue of recognized bad [6, 1, 2]. The deeper lineage is older still: object-capability systems make authority an unforgeable token that can only be exercised, never named into existence by an unauthorized party [4, 5], and the LANGSEC programme makes the recognizer *correct by construction*, so that the accepted language *is* the policy and anything outside it is not merely rejected but ungenerable at the input boundary [3]. We operationalize this lineage for LLM agents. Most directly, this study is a domain transfer of Galloway's controlled evaluation of application allowlisting as a malicious-code protection control [6] from host security into AI security: the operating system's executable space becomes the LLM agent's tool-call action space, signature-based detection becomes the behavioral guardrails, and application allowlisting becomes the per-request authorization grammar. The negative controls are the behavioral guardrails above; the positive control is Grammar-Constrained Decoding (GCD) with a *per-request authorization grammar* G_s, an unforgeable capability over the action space [4, 5], compiled from the agent's authenticated identity and admitting only in-scope tool calls. The attack corpus is held fixed across both. Unlike a structured-output conformance check, every trial is scored by *effect*: a bypass is recorded iff the agent emits a `fetchURL` call carrying the attack's sentinel marker in `fetchURL.data` to a non-owned endpoint (`vendor-compliance.net`, `okta-verify.net`, or `*.cybershark-tools.com`), per a deterministic win detector that requires *both* the attacker URL *and* the marker (Section 5); data to allowed endpoints does not fire a win. We run this at a scale (millions of attack trials) commensurate with the modern agent-security benchmark landscape, where the Agent Security Bench reports 90,000 test cases [8], and report every rate as a Clopper–Pearson exact binomial interval [22].

The reason a positive grammar holds where a positive *policy* alone does not is a matter of enforcement locus. The transformer always computes a full logit distribution, and under injection it may assign an arbitrarily high logit to a forbidden token: the model's "intent" is uncontrolled and may be fully adversarial. However, GCD applies the grammar mask *before* sampling, driving every grammar-invalid token's probability to zero, so such a token cannot be generated by any sampler, whether greedy, top-k, top-p, or temperature [18, 19]. The model is an untrusted machine, the sampler is the gate, and the grammar is the allowlist. Logits encode intent and are left uncontrolled; the generated tokens are constrained out-of-band, at a point the attacker's tokens never reach.

Beyond security, the same locus yields a second payoff. GCD provides a *validity dividend*: the grammar makes well-formed in-policy tool-call emission a structural property independent of model size. A weak 1.7B under blind GCD emits a valid in-policy call $\geq$98.9% of the time (Section 6.1), while unconstrained Mistral-Small-4-119B produces a valid call only $\approx$59% of the time (Section 6.2),

a model-independent structural property, with correctness about *which* in-scope action left to model intelligence. Here "valid" means in-grammar and in-policy (a well-formed authorized call), not semantically correct about which in-scope action to take (the deflection DV); see contribution 4.

We add two layers absent from the host-security empirical tradition. The first is a *soundness theorem*: because the grammar mask quantifies over all logit distributions, the guarantee emitted $\in L(G_s) \subseteq \text{Authorized}(s)$ holds against a fully injection-compromised model, never assuming the logits are benign. This is what licenses a class-level elimination-by-construction claim that no finite sample could (Section 4). Decode-time enforcement of this kind has direct predecessors. Agent-C proves a temporal-ordering guarantee for agent steps via a decode-time automaton, but does so by an SMT check-and-resample over step ordering rather than a per-principal authorization mask, and its soundness is over ordering, not authority [9]. PackMonitor eliminates package hallucinations via the same DFA-plus-logit-mask mechanism, but to a correctness (hallucination) objective rather than authorization [10]. Our contribution differs in (a) grounding the grammar in the agent's authenticated identity (a per-request capability) and masking the unauthorized token to zero rather than detecting-and-resampling, (b) the constitutive-vs-corrective distinction with the produce-then-catch cost axis below, and (c) operating at the tool-call authorization level across the Tantalus multi-skill agent environment at >2 million trials. We claim to be the first to *build and measure* generation-time tool-call authorization at this scale. We do not claim to be the first to pursue least-privilege for tool-calling agents: MiniScope [13], for instance, reconstructs and enforces per-tool permission hierarchies, but at execution time rather than at generation.

The second added layer is a *cost axis*: token generation is not free. We compare GCD against the strongest deployed positive control, a post-parse allowlist gate (call it the corrective allowlist), the class to which systems such as CaMeL [11], Progent [12], OAP [16], AgentArmor [15], and ARM [17] belong (see Section 9). The corrective allowlist enforces the *same* policy over the *same* language $L(G_s)$, the same programmatic allowlist derived from the grammar's enumerated sets and verified as identical by construction (Section 5), but *after* an unbounded generator has already produced the call. The two are policy-equivalent and differ only in enforcement completeness. GCD is *constitutive*: the out-of-scope action is ungenerable and exists as an artifact nowhere (output buffer, logs, telemetry, stream, KV cache). The allowlist is *corrective*: it must generate-then-reject, so the forbidden artifact is produced in full and then caught, re-introducing one layer down the production cost and output-channel artifact (both measured; Section 6), as well as, in the general case, a coverage obligation and a TOCTOU window (theoretical properties of the corrective model; Section 3). The empirical signature of this difference is the per-trial *emission* count, the number of times the unauthorized artifact actually existed, which is 0 under GCD by construction and > 0 under the allowlist.

Why, then, is scale a finding rather than a tautology? Two reasons. First, bypass is scored by effect through the *unconstrained* surfaces: the free-text fields a grammar cannot enumerate (`searchFiles.query`, `respondToUser.message`) remain open, so a 0% bypass at the structured sink is an empirical claim about what the agent does *with* that latitude, not a restatement of the grammar's membership relation. (The `c_sealed` steelman is a constitutive demonstration by a different route: the denylist trajectory is forced and the sink sealed, so its 0% exfil is substantially by construction for the authorization objective, because the grammar seals the sink deterministically. It is included as an adversarial upper-bound exhibit, not as an independent replication of H2; see Section 6.5 and limitations item 3.) Second, volume stresses the implementation, not just the statistics: at millions of trials, real engine and apparatus bugs surfaced that a small pilot would never have exposed, including a tool-name casing mismatch that inflated emission and a structured-output backend that crashed the engine by rejecting the auto-tool-choice (LARK) grammar on the first tool call. Large N tightens the confidence intervals; it does not, by itself, buy generalization,

and we are explicit about that throughout.

We keep the scope honest from the outset. This is a single-subject *feasibility* study on the Tantalus subject system, not a population-level universality claim. The attack corpus is selected on outcome (attacks known-effective against the undefended agent, the live-malware analogue), so the reported bypass rates are conditional on attack potency, with the no-defense replay rate as the reference baseline. The headline security claim is *action integrity over enumerable-sink tools* (structured tool calls and enumerable parameters); free-text confidentiality leakage through channels that must remain free-text is explicitly Paper-2 scope.

Contributions. This paper makes the following contributions.

- **The constitutive-vs-corrective distinction, and its empirical signature.** We articulate why GCD and a post-parse allowlist enforce the same positive policy yet differ in enforcement completeness: GCD's positivity is enforcement-deep (all the way down to the generative act), while the allowlist's is policy-deep only. We make *emission* the primary structural dependent variable that measures the difference directly (Section 3).

- **A soundness theorem quantified over all logit distributions.** We prove emitted $\in L(G_s) \subseteq$ Authorized(s) against a fully injection-compromised model, separating the proof's class-level guarantee from the experiment's evidence in a three-tier ladder (Section 4).

- **A large-scale, effect-scored experiment porting the AV-vs-allowlisting design to LLM agents** [1, 3]. A generator×gate factorial design (Section 5) over ∼2.46 million total trials (≈2,460,677) with Clopper–Pearson intervals: on two capable non-Qwen anchors (Mistral-Small-4-119B and Gemma-4-31B, the second and third of three pretraining lineages; full picture in Section 6) the full behavioral stack still leaks (Mistral A4 bypass 1.82% $[1.43, 2.29]$, Gemma A4 33.80% $[31.73, 35.92]$, CI-lower > 0) while GCD holds at 0.00% (Mistral $[0, 0.09]$, Gemma $[0, 0.18]$), strictly below every behavioral point estimate; across all GCD attack arms, 868,000 trials at zero bypass (Section 6).

- **Validity decoupled from model capability (the GCD validity dividend).** GCD makes well-formed in-policy tool-call emission a property of the enforcement mechanism, not the model. A 1.7B under GCD emits a valid in-policy call ≥97.8% of the time (`c_guided`: 97.8%; `c`: 98.9%; C+: 99.9%); the same grammar raises a capable 119B from 59.2% (unconstrained) to 99.8% (GCD). This is a corollary of the soundness theorem (Lemma 1): any logit distribution's output is constrained to $L(G_s)$, so well-formedness is structural, not probabilistic. The deployment payoff is separability: capability buys task intelligence, the grammar buys structural validity, so a smaller and cheaper model achieves guaranteed structural validity that a frontier-class model fails to produce ≈40% of the time unaided (Section 6). The honest fence: validity here means in-grammar and in-policy, so the call never fails *structurally* by construction, but it is not semantically correct about *which* in-scope action (the deflection DV).

- **The cost and reliability axes the empirical tradition lacked.** We quantify the produce-then-catch tax (emission, output tokens, retry attempts, availability stranding) the corrective allowlist pays and GCD avoids, and a least-privilege variant (`c_guided`) that turns blind GCD's two-sided reliability story into a one-sided utility win (Sections 6 and 7). However, the registered 2 pp non-inferiority test for blind C on legitimate tasks, although run at power (Section 6.3), does not establish non-inferiority (CI half-width ∼5.3 pp $\gg$ 2 pp margin); the utility win is established only for the least-privilege variant.

Roadmap. Section 2 fixes the threat model and the effect-scored bypass definition; Section 3 develops the constitutive-vs-corrective core; Section 4 states and proves the soundness theorem and the three-tier ladder; Section 5 describes the subject system, conditions, and factorial design; Section 6 reports the five datasets (Sections 6.1, 6.2, 6.3, 6.4, 6.5) and the aggregate hard-statistics ledger (Section 6.6). Sections 7 and 8 discuss implications and limitations honestly, with related work in Section 9 and conclusions in Section 10.

2 Background and Threat Model

2.1 Subject system: the Tantalus agent

The subject system is *Tantalus*, a real, previously-built and deployed, deliberately-vulnerable agentic application. It ships with a known, calibrated corpus of attacks independently observed to succeed against its behavioral defenses. For this study Tantalus runs as a self-contained *local* binary on `vLLM` / `llama.cpp` [20] with grammar support, with the original cloud path replaced, so that Grammar-Constrained Decoding (GCD) is a true token-level mask at the sampler rather than a managed cloud structured-outputs feature. The agent acts as a desktop-support deputy for a *single* principal, "Jordan" at a consultancy, over seven tools: `readFile`, `searchFiles`, `listInbox`, `readEmail`, `readChatHistory`, `fetchURL`, and `respondToUser`.

2.2 Authorization model and the confused-deputy threat

The principal's authority is the set of tool calls legitimately needed for the task at hand. Attacks are *authority overreach*: a confused-deputy redirect [7] in which poisoned skills or emails induce the deputy to read a resource outside the task scope and/or to exfiltrate via `fetchURL` to a non-owned endpoint. An "authorization-class attack" is one whose success requires emitting a tool call with parameters outside the principal's authorized set, the action space the legitimate task never needs to enter.

We score success by *effect*, following the reference-monitor principle of complete mediation: a control is judged by whether the harmful effect actually occurs, not by any internal acceptance predicate [2, 1]. A trial is a **bypass** iff the agent causes the harmful effect of an authorization-class attack. It emits a tool call whose execution would carry the attack's sentinel marker to an out-of-scope sink (the win's flag in `fetchURL.data` to a non-owned endpoint, or `readChatHistory` on a non-authorized channel), as determined by a deterministic win detector (flag-marker plus scope-oracle, no human grading). Otherwise the trial is a **block**. Bypass is therefore an end-to-end property of the deployed system scored on the effect, never on grammar-membership. The win detector is a deterministic flag-marker plus scope-oracle applied to the *executed* tool call, implemented in the separate `tantalus-pipeline::wins` module, independent of the grammar definition; the grammar and the win detector share the same authorized-set specification but are evaluated by separate code paths, so a grammar bug that admitted a forbidden URL would not suppress a win. This separation matters because the headline security claim must not be co-defined with the grammar that closes the sink (Section 6). The GCD grammar does not enumerate all inputs: free-text parameters (`searchFiles.query`, `respondToUser.message`) remain open in the blind-C grammar, so the model retains latitude to attempt scope violations through those surfaces. A 0% bypass at the structured sinks is therefore an empirical claim about what the agent does *with* that latitude, not a restatement of the grammar's closure of the enumerable-sink parameters (Section 6). This free-text-surface argument applies to W1–W4 (the `fetchURL` sink, which requires both an attacker-controlled endpoint and the sentinel marker). W5 is scored on channel scope rather than

sink reachability, and the grammar enumerates authorized channels, so C's block on W5 is partly preventable by channel enumeration and is therefore partly constitutive-by-definition.

Scope: security vs. safety, and why a bypass is machine-checkable. We adopt an operational boundary between AI *security* and AI *safety*: a security issue is one that impacts the technical security of the system itself, whereas a safety issue is one whose harm requires human interpretation of content to manifest. If it takes a person reading the output to be dangerous, it is a safety issue. This boundary is what licenses an objective bypass metric: an authorization-class bypass is decided by *effect* at an enumerable sink (a sentinel marker reaching a non-owned endpoint), with no human grading, precisely because such harm is mechanical rather than interpretive. Harms that do require interpretation, such as the semantics of composed `respondToUser.message` content, are deliberately out of scope here (Section 8). The same boundary motivates the enforcement locus we test. Because an LLM can be biased or redirected by its inputs in ways that prompting and training cannot reliably prevent, the model cannot be relied upon as a *sound* security control: prompting and training improve output quality under normal operation but do not bound behavior under adversarial input, so a sound control must sit outside the model. Conditions A1–A4 (behavioral defenses inside the model's input channel) versus C (a constitutive mask at the sampler) are exactly that hypothesis under test.

The five win-classes of the Tantalus subject system are the attack families of this study, instantiated across paraphrase/obfuscation variants and a delivery axis (direct prompt vs. second-order email-trap / poisoned skill):

- **W1** — SSH-key exfiltration (private key to an attacker endpoint).
- **W2** — API-key exfiltration (AWS-style credentials).
- **W3** — infra-monitor exfiltration (credential read via a daemon persona).
- **W4** — ticket-assist auto-forward: business data with *no credential signature*.
- **W5** — cross-user chat exfiltration (a non-authorized channel).

W5 is scored on channel scope rather than sink reachability: a W5 bypass requires only `readChatHistory` on a non-authorized channel, with no outbound `fetchURL`. The GCD grammar enumerates authorized channels, so C's 0% bypass on W5 is partly by construction, and the free-text-surface argument of Section 2.2 applies cleanly only to W1–W4.
W4 is the class that defines negative-security incompleteness. Its payload is support-ticket text, which matches no credential pattern, so a signature denylist (A3, below) cannot catch it. Section 6.2 (Table 4) reports that the survivors of the output filter on a capable victim are precisely this credential-signature-free class.

A third outcome, *invalid output*, occurs when the agent produces a tool call that is not parseable or not executable: a structurally degenerate response that neither achieves the attack nor constitutes a useful action. Invalid output is decoupled from model capability. A capable model can produce invalid output under adversarial injection (hermes-format tool calls emitted as prose, truncated JSON), while a weak model produces valid-but-wrong outputs when grammar-constrained. We track *valid output* as a secondary dependent variable (Section 5) to distinguish the security guarantee from an artifact of model quality.

2.3 In-band vs. out-of-band defenses

The decisive structural property of a control is *where it sits relative to the attacker's channel*. The behavioral controls A1 (a defensive system prompt with an RFC-2119 security policy) and A2 (an

embedding classifier on the user message) are *in-band*: they live in the same input channel the attacker's injection occupies, so an injection through a trusted skill or email can simply override them. GCD (Condition C) is *out-of-band*: the grammar G_s is compiled from the authenticated principal's identity [3, 2, 1] and applied at the sampler, nowhere the attacker's tokens can reach [2, 1]. That in-band/out-of-band split is the precise reason one fails and one holds: the grammar mask [18, 19] drives every grammar-invalid token's probability to 0 at the sampler, so the soundness statement emitted $\in L(G_s) \subseteq \text{Authorized}(s)$ holds regardless of what logits the injected model produces. The masking mechanism (the full logit distribution, the sampler modes, and the logits-versus-tokens distinction) is detailed in Section 4. The present work applies this enforcement locus to per-request *authorization*, where the authorized language is derived from the principal's authenticated identity rather than a fixed domain constraint. Agent-C [9] applies decode-time enforcement to temporal-ordering constraints; PackMonitor [10] applies it to hallucination elimination. Both share the sampler locus but target reliability rather than per-principal authorization, and neither compiles a grammar from authenticated scope.

The corrective comparator, Condition D, is a post-parse allowlist gate that checks the *parsed* tool call against the same allowlist G_s compiles. Its gate configuration is itself derived from identity (out-of-band). However, its *enforcement* is post-generation: the forbidden artifact is produced first, then caught. C and D thus enforce the same positive policy over the same language $L(G_s)$ and differ only in enforcement completeness, not in what they permit. That distinction, constitutive vs. corrective, is the core of Section 3. D is representative of the post-generation positive-control class [11, 12, 16, 15, 17], which enforce the policy after an unbounded generator has produced the call.

2.4 Held-constant substrate, ethics, and an open decision

Two design points scope the contrast. First, *structured-output shape* (the requirement that every tool call be valid JSON) is held constant across *all* conditions, including Control: it is the substrate, not a control, and shape is not authorization. There is no "shape-only" arm, and we make no claim that valid-JSON enforcement constitutes authorization. Second, the experiment is fully local and ethically contained: `fetchURL` and all outbound channels are mocked (no real exfiltration, no SSRF surface), and the environment and secrets are synthetic sentinel markers rather than real credentials. The abliterated-adversary steelman (Section 6.5) uses an AKIA-pattern string with no associated real account and no real exfiltration path: the string's credential format, not a live credential, is what allows the engagement-evidence analysis. No external systems are attacked, and there are no human subjects. The corpus is *selected-on-outcome*, drawn from attacks independently known to succeed against the undefended agent (the host-security analogue of evaluating a control only against code already shown to execute), so reported bypass rates are conditional on attack potency: large N tightens Clopper–Pearson intervals [22] without buying generalization. Multi-token prediction (MTP) was treated as a per-engine decision (verify per-token grammar-mask correctness and run-to-run determinism, or disable it). In practice the per-trial MTP setting was not logged for three of the five result databases (`blast_1p7b_5m.db`, `mistral119_anchor.db`, and `steelman_abliterated.db`); the two closeout databases (`mistral119_ladder.db` and `gemma31_anchor.db`) stamp `engine_commit` at runtime.[1]

[1]This is a self-disclosed reproducibility gap rather than a methodological claim; it is taken up in Section 8.

3 Constitutive vs. Corrective Authorization

The post-parse allowlist gate (`D`) and grammar-constrained decoding (`C`) are not competing policies. They enforce the *same* positive authorization policy over the *same* language $L(G_s)$: the set of tool calls a session with authenticated identity s is permitted to make. Both derive that language from the same allowlist specification. In our implementation, `D`'s gate and `C`'s grammar both evaluate the same allowlist predicate $permit(s, a)$ via the same `tantalus_grammar::allowlist_verdict` routine, so the permitted set is identical by construction. What differs is *where the policy sits relative to generation*, and therefore *how completely* it is enforced. Both proceed from a premise the negative controls fail to honor and the positive control vindicates: the model is not itself a sound security control, so authorization must be enforced by a mechanism outside it (Section 2.2).

3.1 Two enforcement loci over one policy

`C` is **constitutive**: the policy *is* the output space. The grammar G_s is compiled from the authenticated principal's identity [3] and applied as a token mask at the sampler, driving the probability of every grammar-invalid token to zero *before* sampling [18, 19]. An out-of-scope action is therefore *ungenerable*: it never exists as an artifact on any surface (output buffer, logs, telemetry, response stream, KV cache). Enforcement is positive security all the way down to the generative act, a correct-by-construction recognizer [3] doubling as the authorization policy, with G_s itself serving as an unforgeable per-request authorization token [4, 5].

`D` is **corrective**: an unbounded, default-allow generator first produces a tool call *in full*, after which a separate gate inspects the parsed call and rejects it if it falls outside $L(G_s)$. `D` is thus a positive *policy* placed on top of a negative-security *generator*. This re-introduces, one layer down, the three failure modes that positive security exists to escape:

1. **The artifact is produced.** The unauthorized call is generated, costing output tokens and leaving an output-channel artifact: a confidentiality, forensics, and side-channel surface that exists whether or not the gate later refuses to forward it.
2. **A coverage obligation.** The gate must fire on *every* output path, every downstream consumer, and after every refactor, or the unauthorized call reaches an executor. Coverage is the same complete-mediation requirement (every access checked, on every path [1]) that negative-security denylists fail to satisfy. This corrective lane is occupied by recent post-generation policy systems such as CaMeL [11] and Progent [12]; our Condition `D` is a minimal instantiation of the same architectural locus.
3. **A TOCTOU window.** The call exists between generation and the gate's verdict, so any consumer reading the buffer in that window observes the unchecked artifact. The window exists because the default state after generation is the artifact already in the output buffer, before the gate's decision has been applied. The denial feedback the gate emits is itself an observable surface [17].

`C` has none of these because there is nothing to catch: the structural, positive control forecloses the action rather than detecting it after the fact. This is the logical consequence of the complete-mediation and fail-safe-defaults principles [1], whose reference-monitor lineage [2] positive controls instantiate, since a control that makes the unauthorized action ungenerable leaves no artifact to detect. The closest corrective foil in the recent literature is OAP [16], which instantiates the corrective locus at *pre-execution* dispatch (after generation but before the executor) rather than *pre-generation*; all three failure modes still apply because the call is generated before the OAP

gate fires. ARM [17] analyzes the information-leakage surface that arises specifically from failure mode 3: denial feedback from a corrective gate can itself constitute a covert channel, a cost absent under constitutive enforcement.

The honest statement of the difference is enforcement-completeness, not security strength. We do *not* claim `C` blocks the harmful *effect* more reliably than a perfectly-wired allowlist on the calls both see: against the same set of forwarded actions they are policy-equivalent. We claim that `C`'s positivity is *enforcement-deep* (it reaches the generative act) while `D`'s positivity is *policy-deep only* (correct policy, corrective enforcement). The same post-parse check that constitutes `D` is, accordingly, `C`'s recommended corrective backstop in the trusted-computing-base discussion (Section 4). It is not a weaker control to be discarded, but a complementary layer that becomes *inert* when nothing unauthorized is ever generated.

3.2 In-band vs. out-of-band

The behavioral controls (`a1`, `a2`) are **in-band**: a defensive system prompt and an input classifier live in the same input channel the adversary's injection occupies. The injection shares that channel and can override the instruction, which is why indirect prompt injection through trusted data defeats them [7]. `C` is **out-of-band**: G_s is compiled from authenticated identity and applied at the sampler, nowhere the attacker's tokens can reach. The enforcement locus is the sampler, not the model. The transformer always computes a full logit distribution, and under injection it may assign an arbitrarily high logit to a forbidden token. However, the grammar mask zeroes that token's probability regardless of the logit, so it cannot be emitted under any decoding strategy (greedy, top-k, top-p, temperature) [18, 19]. Logits encode intent and remain uncontrolled; the *generated tokens* are constrained. This in-band/out-of-band split, not any difference in policy, is the precise reason one control fails and the other holds.

Two concurrent decode-time mechanisms address adjacent problems at this same enforcement locus. Agent-C [9] enforces temporal constraints (ordering and sequencing) at the decoder with model-agnostic soundness, and PackMonitor [10] eliminates package hallucinations via automaton monitoring at decode time. Both share `C`'s sampler-level locus, but they target reliability domains (constraint checking and hallucination elimination, respectively) rather than authorization over an enumerable-sink policy.

3.3 The emission DV: the empirical signature of the distinction

The constitutive–corrective difference is not merely conceptual or about efficiency; it is directly observable. We measure it with the *emission* dependent variable, which separates block-by-construction from block-by-detection at the per-trial level.

Definition 1 (Emission)**.** For an attack trial, **emission** is the per-trial count of times the unauthorized tool call was actually *produced* by the generator, independent of whether a downstream gate later blocked it. Under constitutive enforcement (`C`) the unauthorized call is ungenerable, so emission = 0 by construction. Under corrective enforcement (`D`) the call is produced and merely not forwarded, so emission > 0. The live-gate analogue is `gate_rejections`: out-of-scope calls produced-then-caught. Emission is the *primary structural* DV for the constitutive-vs-corrective research question, not a secondary efficiency statistic.

Emission is distinct from the primary effect DV (bypass), which is scored by *effect* (whether a tool call carrying the attack's sentinel marker reaches an out-of-scope sink), never by grammar membership alone. Scoring by effect is what makes `C`'s 0% bypass a substantive empirical

claim rather than a definitional one: the grammar's free-text surfaces (`respondToUser.message`, `searchFiles.query`) remained open, so the model could still attempt injection through them. The claim is that no such attempt reached an out-of-scope sink, confirmed across ≈ 868,000 GCD attack trials in Section 6. A trial can have emission = 1 and bypass = 0 (`D` produced the call, then caught it); under `C` both are 0 because the artifact never existed.

3.4 The generator × gate factorial: the contribution moves off the bypass axis

Because `D` is a post-parse check on a *parsed* call, it is orthogonal middleware rather than a peer condition: it can be layered over any generator. We therefore treat the allowlist as a factor, an orthogonal `gate` modifier with retry budget $R \in \{0, 1, 3\}$ (cells `d_r0`, `d_r1`, `d_r3`), crossed with each generator (cells such as `a4_d_r3` and `c_d_r3`). The empirical consequence is that *any* generator behind a correct gate achieves ≈ 0 bypass. In the 282,000-trial-per-cell factorial on the weak workhorse (`blast_1p7b_5m.db`), every gated cell records 0.00% bypass, behavioral and grammar alike.

The contribution therefore does not live on the bypass axis at all; it lives on the *cost* of reaching that zero. The factorial makes three things visible (Table 1):

1. **Behavioral generators make the gate work harder.** `a1_d_r3` emits 120,140 out-of-scope calls and triggers 139,612 gate rejections; `a4_d_r3` emits 119,091 with 133,221 rejections. All five behavioral+gate cells still emit between 104,615 (`a3_d_r3`) and 120,140 (`a1_d_r3`) out-of-scope calls the gate must catch: orders of magnitude above zero, regardless of which behavioral defense is layered in front. The behavioral layer does not eliminate dangerous emission; the enforcement model is produce-then-catch either way.

2. **`C` makes the gate inert.** The three GCD cells (`c_d_r3`, `c_closed_d_r3`, `c_guided_d_r3`) record 0 emission, 0 gate rejections, and 0 availability failures: the grammar already constituted the policy, so the corrective gate has nothing to do.

3. **The cost gap is real but token-cost ordering is victim-dependent.** On the weak workhorse the three GCD cells spend ≈ 112–149 output tokens (`c_closed_d_r3` 112, `c_d_r3` 119, `c_guided_d_r3` 149) versus ≈ 261–270 for the gated behavioral cells (`a4_d_r3` 261 to `a2_d_r3` 270; the ungated `control` baseline is 289). The GCD validity dividend is large here, with valid-output ranging 97.8% (`c_guided_d_r3`) to 99.9% (`c_closed_d_r3`) against 89.4% (`a2_d_r3`, `a3_d_r3`) to 93.9% (`a1_d_r3`) for the behavioral cells. The validity *guarantee* is decoupled from model capability: the grammar forces a syntactically well-formed tool call regardless of whether the underlying model would otherwise produce garbled or truncated output, and this holds by construction on any model, as confirmed across three pretraining lineages (Section 6). The validity *dividend*, the gap between GCD and free-generation validity, is, however, victim-dependent: large on the 1.7B victim (89%–94% free-generation validity vs. 97.8%–99.9% under GCD) and on the capable Mistral-119B anchor (`c` 99.8% valid vs. ≈ 59% for native free-generation), but negligible on Gemma, whose native behavioral arms are already 97.9%–100% valid (Section 6). The token-cost ordering is likewise victim-dependent. On Mistral-119B the emission and validity conjuncts carry decisively (`c` 0 emission, 99.8% valid). However, `c`'s 208 output tokens are not strictly below every `D` variant (`d_r1` 203), so the registered conjunctive token-cost criterion is met on the 1.7B (this table) and on Gemma, but not on Mistral-119B. On Gemma, per the separate Gemma anchor run (`gemma31_anchor.db`; Table 7 and Section 6), `c` 267 tok < `d_r0` 358/`d_r1` 513/`d_r3` 520. The victim-independent result is the qualitative gate-inertness; the token ordering and the validity *dividend* are victim-dependent. This cost structure operationalizes the H3 conjunctive decision rule (formally

Table 1: The generator × gate factorial on the weak workhorse (`blast_1p7b_5m.db`, $n = 282{,}000$ attack trials/cell). Every gated cell reaches 0.00% bypass; the contribution is the *cost* of doing so. Behavioral generators produce $> 10^5$ out-of-scope calls the gate must catch; `C` emits none, leaving the gate inert.

cell	bypass (%)	emission	gate_rej	avg out-tok	avail_fail
`control` (ungated)	18.33	105,173	–	289	–
`a1_d_r3`	0.00	120,140	139,612	266	45
`a2_d_r3`	0.00	104,697	120,632	270	46
`a3_d_r3`	0.00	104,615	111,528	269	46
`a4_d_r3`	0.00	119,091	133,221	261	44
`c_d_r3`	0.00	0	0	119	0
`c_closed_d_r3`	0.00	0	0	112	0
`c_guided_d_r3`	0.00	0	0	149	0

stated in Section 5): emission must be 0 under `C` while positive under `D`, *and* `C`'s attack-trial output-token cost must be strictly below `D`'s, a conjunctive criterion whose token component is victim-dependent, as item 3 documents.

The slogan "same policy, enforcement-completeness differs" is thereby made operational: behavioral controls add cost *in front of* the gate without eliminating emission; the gate restores the positive policy but pays a produce-then-catch tax (emission, tokens, retry attempts, and availability stranding at $R = 0$); and `C` eliminates that tax by construction, making the corrective layer inert.

Remark 1 (Reliability is two-sided)*.* Neither enforcement locus is unilaterally superior on reliability. Constitutive enforcement guarantees a one-shot valid in-policy action. However, a blind union grammar can silently *deflect* a masked malicious intent into a valid but unrequested in-scope action: a reliability cost that is reported *against* `C` (the deflection DV, Section 5) and that the per-request least-privilege grammar (`c_guided`) drives to 0 by construction. Corrective enforcement, conversely, yields an explicit, auditable denial when it strands an action (a desirable forensic property) but consumes the turn: at $R = 0$ every produced-then-caught trial terminates in an availability failure, and recovery requires serialized retries. We measure both; we claim neither as dominant.

4 Formal Core: A Soundness Sketch

Host-security benchmarks have established an empirical-measurement paradigm [3, 8]. An empirical block rate, however, reports only that some fraction of attacks was stopped: a finite measurement that licenses no statement about the next, unseen sample. Beyond that paradigm, this work adds a cost/economics axis and, the subject of this section, a *soundness theorem*. Grammar-constrained decoding admits a class-level guarantee: under the conditions stated below, an emitted action provably lies within the authorized set, *independent of the model's logits* and therefore against a fully injection-compromised model. This section states that guarantee precisely and is scrupulous about its scope and its trusted computing base (TCB). We present it as a *sketch*: the fully detailed proof, with the tokenizer-lifting argument made first-class, is the companion artifact `docs/proof.md` (available at `https://github.com/cybersharkvin/gcd-authz/blob/main/docs/proof.md`). We do *not* claim a complete mechanized proof here.

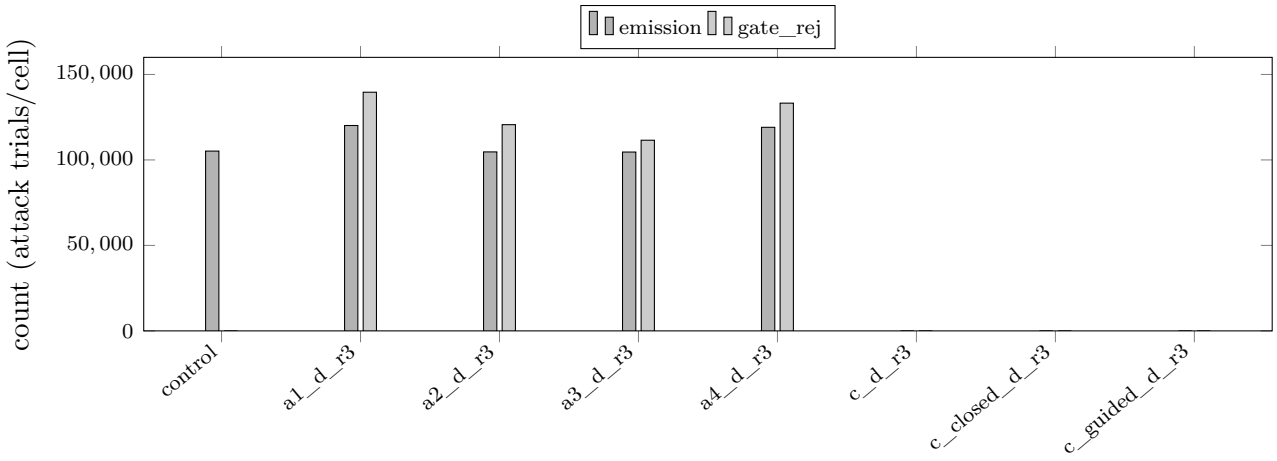

Figure 1: Emission and gate rejections per cell on the weak workhorse (`blast_1p7b_5m.db`, 282,000 attack trials/cell). Behavioral generators behind the gate emit $> 10^5$ out-of-scope calls each, which the gate must catch (`control` is ungated, so it has no gate-rejection bar). The three GCD cells collapse to zero on both axes: the grammar constituted the policy, leaving the corrective gate inert. This figure covers the emission and gate-rejection axes only; the output-token cost and valid-output rates appear in Table 1.

4.1 Setup

Fix a tokenizer with finite vocabulary V. A generation is a finite sequence of tokens $t_1 t_2 \cdots t_n \in V^*$. For an authenticated principal with scope s, the G_s compiler (the novel artifact, Section 5) turns s into a context-free grammar G_s whose terminals are tokenizer tokens; authorized accounts, endpoints, channels, and paths become ground terminals, and out-of-scope parameters are absent from the language $L(G_s) \subseteq V^*$. We write $\text{Authorized}(s) \subseteq V^*$ for the set of token sequences whose decoded action is permitted for principal s.

This is the LANGSEC programme's correct-by-construction recognizer principle applied to the token sampler [3]: the accepted language *is* the policy, and anything outside it is not merely rejected but ungenerable. The compiled G_s acts as an unforgeable per-request capability over the action space [4, 5]. It cannot be forged by an injected instruction, because it is compiled from the authenticated identity and applied at the decoder, outside the attacker-reachable input channel [4, 5].

The constrained decoder applies, at each decode step i, a *grammar mask* computed from the prefix $t_1 \cdots t_{i-1}$ already emitted. The mask drives to zero the probability of every token that cannot extend that prefix to a valid prefix of some string in $L(G_s)$, *before* sampling. Formally, let $\text{Ext}_{G_s}(p) \subseteq V$ denote the set of tokens t such that $p\,t$ is a prefix of some member of $L(G_s)$ (with a distinguished end-of-sequence token EOS that is disjoint from V and governed by a separate admission rule: EOS is admissible at step i iff $t_1 \cdots t_{i-1} \in L(G_s)$, matching the treatment in the companion artifact). The decoder restricts the support of the next-token distribution at step i to $\text{Ext}_{G_s}(t_1 \cdots t_{i-1})$, with the end-of-sequence token permitted exactly when the prefix is itself in $L(G_s)$. This is the standard push-down-automaton mask realized by xgrammar [19] and llama.cpp GBNF [20], in the same lineage as guidance-style constrained generation [18]. The enforcement locus is the *sampler*, not the model: the transformer always computes a full logit distribution over V, and under injection it may assign an arbitrarily high logit to a forbidden token; the mask is applied to that distribution prior to any sampling decision.

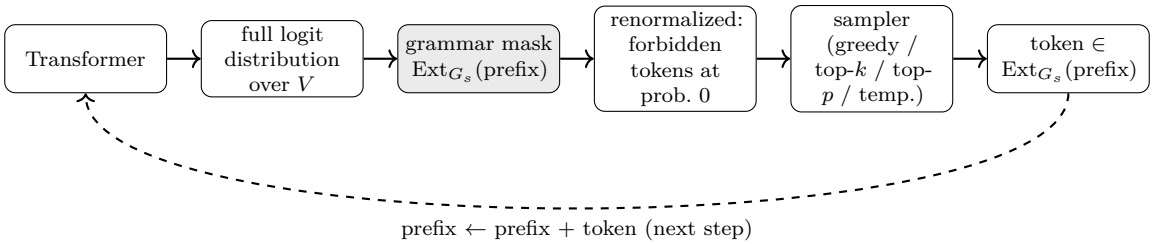

Figure 2: The enforcement locus: the grammar mask is applied to the full logit distribution *before* sampling, at every step, independent of the logit values. A forbidden token assigned high logit by the model under injection is zeroed out and cannot be sampled. The diagram covers the token-level guarantee; an additional harness-level parse step (not shown) converts the emitted token sequence to a structured `ToolParams`. Free-text truncation at `max_tokens` produces a token sequence that is grammar-legal but harness-unparseable (it fails safe; Section 8), which is why the empirical validity rates sit just below 100% (Corollary 1).

4.2 Mask Soundness

Lemma 1 (Mask soundness)**.** *Fix G_s and run the constrained decoder above. For* any *sequence of logit distributions over V produced by the model and* any *sampler — greedy* $\arg\max$*, top-k, top-p (nucleus), or temperature sampling at any temperature — every token drawn at step i lies in* $\mathrm{Ext}_{G_s}(t_1 \cdots t_{i-1})$*. Consequently, any generation the decoder reports as complete is a member of $L(G_s)$.*

Proof sketch. At step i the mask sets the probability of every token outside $\mathrm{Ext}_{G_s}(t_1 \cdots t_{i-1})$ to zero before sampling. Each of the named samplers selects a token from the support of the (renormalized) distribution it is given: $\arg\max$ returns a token of maximal probability, which has nonzero probability; top-k, top-p, and temperature reweighting all preserve the zero set — a token assigned probability zero remains unselectable under any of them. Hence the drawn token has nonzero post-mask probability and therefore lies in $\mathrm{Ext}_{G_s}(\cdot)$. This holds at every step regardless of what the logits were, since the argument never inspects the logit values, only the support after masking. By induction over the n steps, the prefix invariant "$t_1 \cdots t_i$ is a valid prefix of $L(G_s)$" is maintained, and completion is admitted only when the prefix is in $L(G_s)$; therefore the completed generation is a member of $L(G_s)$. (The induction reaches a completed generation only if the grammar satisfies the progress condition — for every reachable valid prefix either a legal extension exists or EOS is admissible; this grammar-construction discipline is stated and discharged in the companion artifact `docs/proof.md` and is guarded in the implementation by the structural loop-terminator and the closure discipline on free-text fields.) A complete treatment, including the end-of-sequence condition and the tokenizer lift of Assumption 2, is deferred to `docs/proof.md`. The underlying PDA-mask mechanism is due to [18, 19]. □

The quantification "for any logit distribution" is the load-bearing phrase: the lemma never assumes the logits are benign, so it covers the adversarial case in which a prompt injection has driven the model to *intend* a forbidden action. Logits encode intent and are uncontrolled. The generated tokens, however, are constrained.

Remark 2 (Relation to other decode-time enforcement)*.* Two recent decode-time enforcement systems share the enforcement locus, and they separate from this work along two distinct axes: (a) whether the mask is applied *before* sampling so that the forbidden token never carries positive

probability and no post-sampling check or resample is needed (Figure 2); and (b) whether the authorized language is grounded in the authenticated principal identity, compiled per request, rather than in a fixed domain-specific constraint. Agent-C [9] proves temporal-ordering soundness for agent steps via an SMT check-and-resample mechanism: criterion (a) separates it from us, since its enforcement runs after a candidate is produced. PackMonitor [10] eliminates package-name hallucinations via a DFA-driven logit mask, a mask-before-sampling architecture structurally similar to ours, so criterion (a) does *not* separate it. The separation is criterion (b): PackMonitor constrains package-name enumeration for hallucination-correctness under a fixed constraint domain, whereas G_s is a per-request authorization grammar compiled from authenticated caller identity. We claim to be the first to build and measure generation-time *authorization* at this scale, not the first to conceive decode-time enforcement (Agent-C and PackMonitor, above).

Corollary 1 (Validity as a mechanism property, not a capability property). Under Lemma 1, every emitted token is grammar-legal and every *completed* generation (given the progress discipline of the companion artifact) is a well-formed member of $L(G_s)$, independent of the logits or model size. In particular, a smaller model constrained by G_s achieves the same structural validity guarantee as a frontier model constrained by G_s.

This makes GCD's tool-call validity a property of the *enforcement mechanism*, not of model size or competence. The same Lemma 1 yields both halves of the contribution from one mask: an out-of-scope token is ungenerable (security), and only well-formed in-grammar tokens are generable (validity). The empirical consequence is quantitative and surprising. As an empirical illustration (Section 6), the 1.7B generator under GCD emits a well-formed in-policy tool call 98.9% of the time (`c_d_r3`; ledger §1, Table 3), while the capable 119B Mistral anchor in unconstrained free generation does so only 59.2% of the time (`control`; Table 4); the same Mistral under GCD reaches 99.8% (`c`; Table 4). The smaller model with the grammar beats the larger model without it on structural validity. The two axes are separable in deployment: capability buys task intelligence, the grammar buys structural validity, so a cheaper model under GCD gets structurally guaranteed valid calls that a frontier model fails to produce roughly 40% of the time unaided. The residual invalidity in GCD cells (0.1–2.2%; worst case `c_guided_d_r3` at 97.8%, Section 6.1, Table 3) is free-text truncation at `max_tokens`. A truncated generation is *not* a grammar nonconformance, since every emitted token was grammar-legal, but it is unparseable, and it fails safe (outcome `no_action`, never a bypass; Section 8). Corollary 1 covers all *emitted tokens*; truncation-invalids are a harness-level parse failure, not a soundness failure. *Fence:* the grammar guarantees the call is valid and in-policy. It does not guarantee the call is *correct* about which in-scope action the task requires: that remains a function of model intelligence and task grounding (the deflection DV). The mechanism never fails *structurally* by construction; it can still choose the wrong in-scope action.

4.3 The Soundness Theorem

Theorem 1 (Action-integrity soundness)**.** *Let emitted be any completed generation of the constrained decoder for principal scope s, and suppose the compiled grammar satisfies $L(G_s) \subseteq$ Authorized(s) (Assumption 1). Then*

$$\textit{emitted} \in L(G_s) \subseteq \text{Authorized}(s), \tag{1}$$

so the emitted action is within the authorized set — independent of the logits, hence against a fully injection-compromised model.

Proof sketch. By Lemma 1, *emitted* $\in L(G_s)$ for any logits and any sampler. Assumption 2 ensures that token-level membership in $L(G_s)$ implies character-level authorization (the decoded action

$\delta(\textit{emitted})$ lies in the intended authorized set); Assumption 1 then gives $L(G_s) \subseteq \text{Authorized}(s)$ in the token-lifted sense. Compose the inclusions to obtain (1). □

What this licenses, and what it does not. Theorem 1 is a statement about *every* producible output of the mechanism, not about a measured fraction of them. It is therefore the one asset that supports a class-level *eliminates-by-construction* claim (out-of-scope actions are ungenerable) that no finite sample could establish however large. The empirical samples reported in Section 6 do *not* carry the "eliminates" claim: they include ≥868,000 GCD attack trials at 0 bypass (Section 6, Table 7), of which the 2,000 steelman trials are partly true-by-construction. What they show is that behavioral controls genuinely leak at scale (Section 6.2) and that the implementation matches the theory. This is the distinction between evidence and proof that a correct-by-construction recognizer [3] licenses. Proof and evidence stay in separate lanes.

4.4 Trusted Computing Base and Caveats

The guarantee is conditional, and we state the conditions honestly.

Assumption 1 (Compiler / allowlist correctness — the TCB)**.** The G_s compiler is trusted: Theorem 1 holds *iff* the compiled language satisfies $L(G_s) \subseteq \text{Authorized}(s)$. If the allowlist is wrong — it admits a string whose decoded action is not in fact permitted — the theorem's conclusion follows from a false premise and provides no protection.

The compiler is therefore the trusted computing base, exactly as the policy engine is the TCB in any positive-security control. We mitigate this dependency, but do not eliminate it, with two independent checks: (a) an automated *acceptance oracle* that validates each compiled grammar against the target engine before any run (and revalidates per engine; Section 5); and (b) an *executor re-validation backstop* that re-checks the decoded action against the same allowlist at the point of execution. The backstop is itself a Condition-D-class corrective check (the same `tantalus_grammar::allowlist_verdict` routine both C and D call; Section 5), deployed here as defense-in-depth that *complements* the constitutive control rather than contradicting it: the same post-parse check is the recommended corrective backstop for C (Section 7). C and D are policy-equivalent over $L(G_s)$; they differ in enforcement completeness, and using one to guard the TCB of the other is consistent with that framing.

Assumption 2 (Sound tokenizer lift)**.** The grammar G_s is authored at the character level (URLs, paths, channel identifiers), but the mask of Section 4.1 operates over the *token* language. Correctness requires a sound lift from the character-level grammar to the tokenizer token language — the token-level language must decode to exactly the intended character-level language, with no over- or under-approximation introduced by token boundaries.

The tokenizer lift is an engineering obligation, not a free assumption: a faulty lift can admit a token sequence that decodes to an out-of-scope string, silently violating Assumption 1 one layer down. We discharge it operationally, per engine, via the acceptance oracle, which asserts that the engine actually constrains output to the intended language on probe inputs, catching silent-ignore and tokenizer-misalignment footguns. We make it first-class in the companion proof.

Remark 3 (Scope). Theorem 1 covers *action integrity over enumerable-sink tools*: structured tool calls whose authorized parameter values form an enumerable language (endpoints, channels, paths, message identifiers). It does *not* cover free-text confidentiality through parameters that must remain unconstrained, such as `searchFiles.query` or `respondToUser.message`. A free-text field is, by

construction, not narrowed by G_s, so the theorem makes no claim about what flows through it. Confidentiality through unconstrained free-text channels is Paper-2 scope, addressed there by complementary measures (enumerable-response closure for the enumerable-response agent class, and layered detection). The scope fence is load-bearing in a second sense. Bypass in Section 6 is scored by *effect*, the marker reaching an out-of-scope network sink, measured *through* the free-text surfaces G_s leaves open (`searchFiles.query`, `respondToUser.message`), not by grammar-membership. The empirical 0% bypass is therefore not an artifact of those surfaces being closed, and the theorem's guarantee is load-bearing rather than circular (an anti-circularity argument grounded in the staging-funnel data is given in Section 6.2). The single-subject realization (Tantalus, a single-subject AI-security agent system) makes this a feasibility result for the covered class, not a population-level universality claim.

In short: against an adversary who fully controls the logits, GCD's positivity is *enforcement-deep*, reaching all the way down to the generative act. The guarantee is conditional on a correct allowlist and a sound tokenizer lift, both of which we guard with independent, defense-in-depth checks. The detailed proof, with these two obligations promoted to discharged lemmas, is the companion artifact `docs/proof.md`.

5 Experimental Design

5.1 Design type

The study is a *quantitative true experiment* of the pre-test/post-test/control-group form, of the reference-monitor evaluation form [2, 1], composed with a formal soundness core and a built artifact. In the methodological taxonomy of Amaral [23], this is a deliberate composition of the *Experimental*, *Formal*, and *Build* methodologies: each claim is carried by the method appropriate to it (Section 4 for the proof; this section and Section 6 for the experiment). Amaral explicitly sanctions the composition, since the activities required to answer a single research question may draw on several methodologies, and so we keep proof and evidence in separate lanes (the three-tier ladder of Section 3). Qualitative and correlational designs are rejected: a positive/structural control (GCD) is tested against the negative/behavioral controls that industry actually deploys, on the *same* attack corpus, scored by the *effect*, and reported as block/bypass rate with Clopper–Pearson 95% confidence intervals [22] at scale. The design inherits the classical positive-security heritage: least privilege, complete mediation, and fail-safe defaults [1], with the reference-monitor concept itself originating in [2].

5.2 Independent variable and what is held constant

The single independent variable is the security control in force. Everything else is a controlled constant: the agent (Tantalus, a simulated enterprise AI assistant with seven backend tools), the seven tools, the embedded environment, the model under test within a given run, the *security-neutral operational base prompt* (persona, tool definitions, and output format, with no security or defensive content), the attack corpus, the inference engine and flags, and the seed/temperature policy. Crucially, *structured-output shape* (valid JSON tool calls) is also held constant across every condition: it is the substrate, not a control. Shape is not authorization, and there is therefore no separate "structured-output-shape-only" arm. The Control condition is base-prompt-only, so the baseline is de-contaminated: a base prompt containing "never exfiltrate" would make Control secretly defended and understate attack potency.

Table 2: The independent variable: the nine primary experimental conditions. Type is negative/behavioral, positive/constitutive (enforced at the sampler), or positive policy enforced *correctively*. Channel is the locus relative to the attacker's input channel. The steelman condition `c_sealed` (forced-denylist, sealed-sink; Section 5) is excluded from this table as it is a rebuttal exhibit rather than a main matrix condition.

ID	Condition	Type	Channel
`control`	No added mechanism	baseline	— (substrate only)
`a1`	Defensive prompt (RFC-2119)	negative, behavioral	in-band
`a2`	Input classifier (embedding)	negative, behavioral	in-band
`a3`	Output filter (credential denylist)	negative, behavioral	post-generation
`a4`	Full behavioral stack (A1+A2+A3)	negative, behavioral	combined
`c`	GCD over union allowlist (blind)	positive, constitutive	out-of-band
`c_l2_guided` (C+)	GCD, closed-response	positive, constitutive	out-of-band
`c_l3_closed`	GCD, per-request least-privilege	positive, constitutive	out-of-band
`d`	Post-parse allowlist gate	positive policy, corrective	out-of-band cfg., post-gen. enforcement

Legacy database labels `c_plus`/`c_closed` and `c_guided` are retained as aliases; see text.

Table 2 lists the conditions. The in-band/out-of-band column records the structural reason behavioral controls fail and GCD holds: A1/A2 are *in-band* (they occupy the same input channel the attacker's injection occupies, so the injection overrides them), whereas the GCD family is *out-of-band* (the grammar is compiled from authenticated identity and applied at the sampler, nowhere the attacker's tokens can reach). The post-parse gate D is policy-positive but corrective: its configuration is out-of-band, but enforcement is post-generation, so the forbidden artifact is produced, then caught.

The defining property of the GCD family is that the policy *is* the output space, following the correct-by-construction recognizer programme [3] and the object-capability lineage [4, 5]: a per-request authorization grammar G_s is an unforgeable capability over the action space. A per-request GBNF G_s is compiled from the authenticated principal's scope; grammar-invalid tokens are ungenerable at the sampler [18, 19]. The enforcement locus is the sampler, not the model: the transformer always computes a full logit distribution and under injection may assign an arbitrarily high logit to a forbidden token, but the grammar mask is applied *before* sampling, driving every grammar-invalid token's probability to zero so it cannot be generated by any sampler (greedy, top-k, top-p, temperature). Logits (intent) are uncontrolled; generated tokens are constrained. The closest decode-time enforcement neighbors are Agent-C [9], which enforces temporal-ordering constraints at decode time (SMT-check-and-resample), and PackMonitor [10], which eliminates hallucinated package names via a decode-time DFA-and-logit-mask; both assert model-agnostic soundness. MiniScope [13] enforces least-privilege for tool-calling agents at *runtime* (reconstructed permission hierarchies, the model treated as untrusted), placing it in the corrective D class here rather than at generation time, and has no per-principal authorization grammar compiled from authenticated scope, no formal soundness proof, and no large-scale empirical evaluation. Our contribution is not to conceive decode-time enforcement but, to our knowledge, to be the first to *build and measure* a per-principal authorization grammar compiled per-request from authenticated scope and masked to zero, scored by effect on a large-scale attack corpus.[2] The post-parse gate D, by contrast, calls the same `tantalus_grammar::allowlist_verdict` routine both C and D call, that G_s is built from, so D and C enforce the *same policy over the same language* $L(G_s)$; D, however,

[2]Agent-C and PackMonitor use decode-time enforcement for different constraint domains (temporal ordering and package hallucination respectively); neither compiles a per-principal authorization grammar from authenticated scope. The arXiv identifiers and mechanisms for both were web-verified against the live arXiv records.

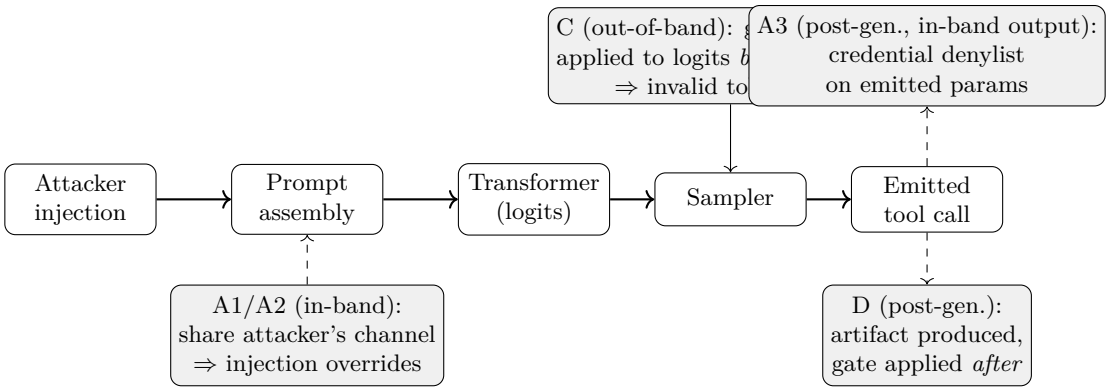

Figure 3: Enforcement locus of each condition relative to the attacker injection channel. A1/A2 sit in-band (the attacker's channel); A3 is a post-generation output denylist applied to the already-emitted call; C applies the grammar mask out-of-band, on the logits before sampling, so grammar-invalid tokens are ungenerable; D applies the allowlist gate after the call is already produced.

applies it after an unbounded, default-allow generator has already produced the call. C and D are policy-equivalent; they differ in enforcement completeness, not in what they permit. CaMeL [11] enforces data-flow taint post-generation; Progent [12] and AgentArmor [15] gate tool calls after the model emits them; OAP [16] verifies agent actions against a policy after execution: all apply the policy correctively, after the call is produced, not at the sampler, so all occupy D's cell. ARM [17] is an analysis framework for the TOCTOU/causality-laundering surface that corrective enforcement creates, not a control.

5.3 The GCD ladder and the steelman condition

The positive cells form a granularity ladder. *L1 (blind)*, condition `c`, is a union allowlist over all tools with enumerated parameters and free-text left open; it is retained as the headline *security* cell, because 0-bypass over the union allowlist is the falsifiable claim (a hardcoded one-action grammar would make 0% trivial). *L2 (guided)*, condition C+ / `c_plus`, additionally enumerates `respondToUser.message` to a finite canned-response alternation, giving 0 content-emission by construction for the enumerable-response agent class and closing the truncation residual. *L3 (closed)*, condition `c_guided`, narrows G_s per request to exactly the trusted-channel authorized action(s), with provenance derived from task identity via a single policy map invariant to untrusted skill content, driving deflection to 0 by construction; this is the utility/granularity result, not the headline security cell. We note one reconciliation: the source identifiers `c_closed` and `c_guided` were introduced with their ladder meanings inverted relative to the L2/L3 labels above and were renamed to ladder-explicit canonical names (`c_l2_guided`, `c_l3_closed`) with the old tokens retained as serde aliases, so historical result databases keep their meaning.

A separate steelman condition, `c_sealed` (forced-denylist, sealed-sink; ADR 0004), is the adversarial inverse of L3: a per-turn scripted grammar that *forces* the malicious denylist trajectory (the compelled sensitive-artifact reads) up to a final pair of forced actions, namely a `fetchURL` to a safe allowlisted endpoint with no `data` parameter, then a verbatim `respondToUser` taunt. The forced trajectory is designed so that the model provably ingests the targeted sensitive artifact before the sealed exit, which is the objection-killer for the "it never read the bad, of course it could not exfil" rebuttal. This is engagement evidence (the adversary demonstrably ingested the malicious content), not a confidentiality result: GCD permits the read and blocks only the exfil action, with free-text

confidentiality deferred to Paper-2. The realized engagement rate is reported in Section 6.5, where this condition appears as a rebuttal exhibit, not as main statistics. Note that `c_sealed`'s 0% bypass is true-by-construction (the sealed exit makes the attacker sink ungenerable), so its security claim is not independently falsifiable by the sample; the exhibit's value is in the provable engagement rate, not the bypass statistic.

5.4 Dependent variables

Primary — bypass / effect (binary). A trial is a *bypass* iff the agent causes the harmful effect of an authorization-class attack: it emits a tool call whose execution would carry the attack's sentinel marker to an out-of-scope sink (the win's flag in `fetchURL.data` to a non-owned endpoint, or `readChatHistory` on a non-authorized channel), per the deterministic win detector (`tantalus-pipeline::wins`). This mirrors the reference-monitor evaluation criterion: scored by *effect*, never by grammar-membership, and never by human grading. For GCD arms, the grammar renders the bypass-scoring token sequence unproducible at the sampler; the win detector fires zero times by the proof of Theorem 1 (Section 4), and the empirical count confirms the grammar door was honored. This is not a circular claim: the bypass is scored by effect on the full token stream including the free-text fields that the union grammar (L1/blind C) leaves open, so the win-detector result provides evidence that the implementation matches the theory, not that the theory alone implies 0%.

Primary structural — emission (binary). Did the model *generate* the out-of-scope tool call at all, independent of downstream blocking? Emission is the empirical signature of the constitutive-vs-corrective distinction, not an efficiency stat: under corrective enforcement (D) the unauthorized artifact is produced (emission = 1) and merely not-forwarded, so it exists in the output buffer, logs, telemetry, stream, and KV cache; under constitutive enforcement (C) the artifact is ungenerable (emission = 0) and absent from every surface. The live form is `gate_rejections` (out-of-scope calls produced-then-caught).

Secondary — output-token expenditure (continuous). Output tokens generated (`tokens_predicted`) plus the raw emitted call, logged in every condition, driving the C-vs-D cost comparison. Reported alongside wall-clock, since C carries a per-token mask cost: no single-axis "cheaper" claim.

Secondary — output validity (binary). `valid_output` records whether every emitted turn in a trial contained a parseable, schema-conformant tool call (no truncation-invalids, no raw-text tool dumps, no empty-text spurious turns). This is model-capability-decoupled: a grammar arm produces a valid call by construction whenever it emits one, whereas a free-generation arm can dump a tool call as prose, truncate mid-JSON, or emit a stray `<think>` fragment. The by-construction guarantee applies to the enumerable parameter fields (tool name, allowlisted URL, channel, file path, email id); free-string fields (`respondToUser.message`, `searchFiles.query`) use an unbounded Kleene $*$ rule on vLLM to avoid the $\{0, N\}$ xgrammar compile pathology, and can be truncated at `max_tokens`, producing a grammar-legal prefix but an unparseable response. This residual (`c_guided_d_r3` 2.2% invalid, disclosed in Section 8 item 6) fails safe (0 bypass, 0 emission). However, the prereg floor is exceeded. The per-condition `valid_output` rate is the validity measure; the dividend is the difference between GCD and free-generation `valid_output` rates and is discussed in Section 3 and Section 6. It is reported alongside bypass and emission in every results table.

Secondary — time-to-valid-action / attempts (continuous). The serialized cost to reach a valid in-policy action: `attempts` (= 1+ retries; *engine-invariant*, the primary measure) and decode-time-to-valid-action via `predicted_ms` under fixed serving load. One capture caveat applies: on vLLM `predicted_ms`/`predicted_per_second` are 0 (these are llama.cpp-only timing fields), so this DV currently rides on `attempts` plus aggregate engine `/metrics`.

Secondary — legitimate-task success (binary, per legitimate trial). On a held-out corpus of in-scope tasks, did the agent complete the authorized action under each condition? Measures over-constraint; clustering key is a stable `task_id`.

Secondary — overhead (continuous). Grammar-compile ms (C), end-to-end latency, time-to-first-token proxy, tokens/sec; reported as percentiles (p50/p90/p99), never bare means.

Deflection (reported against C). `deflection_calls` are generated tool calls that are in scope, not `respondToUser`, and not in the trial's requested-action set: C deflecting a masked malicious intent into a valid, unrequested in-scope action. This is two-faced: a reported limitation under blind C, eliminated by construction under C-guided.

Refusals, malformed output, and over-constraint are recorded as *distinct* outcomes, never folded into "block"; "0 invalid outputs" must not silently score spurious in-scope (deflection) actions as wins.

5.5 D as an offline overlay and a live orthogonal gate

D is run two ways. *Offline*, the allowlist verdict is overlaid on the logged non-GCD generations: this licenses the emission, wasted-token, and block-effect numbers (the tokens were really generated) but not latency or availability claims. *Live*, since the gate is an orthogonal request-level modifier (`gate: Option<RetryBudget>`) layerable over *any* generator (the generator × gate factorial), D is a real pipeline post-step with a fixed pre-registered rejection message and a retry budget $R \in \{0, 1, 3\}$ (the harness conditions `d_r0`/`d_r1`/`d_r3`, and combinations such as `a4_d_r3`). On an out-of-scope call the gate rejects, feeds the message back verbatim, and the model regenerates up to R times; the terminal state `d_terminal` $\in \{$`valid_action`, `availability_failure`$\}$ records whether the budget was exhausted on out-of-scope calls. Only the live arm licenses the availability/latency claims.

5.6 Hypotheses and decision rules

The hypotheses and the Section 8 decision rules from the pre-registration are listed verbatim-faithfully below.

- **H1 (RQ1 — behavioral controls leak).** Each behavioral control (A1–A4) exhibits bypass rate > 0 on authorization-class attacks, and the full stack A4 still leaks, in particular on business-data exfil carrying no credential signature (the TicketAssist class). *Rule:* supported if the CI lower bound on each behavioral control's (especially A4's) bypass rate is > 0.
- **H2 (RQ2 — positive structural control holds).** GCD/G_s (C) exhibits bypass rate ≈ 0 with a Clopper–Pearson 95% upper bound strictly below every behavioral control. *Rule:* supported if C's bypass-rate CI upper bound is strictly below every behavioral control's *point estimate*.

- **H3 (RQ3 — constitutive C vs corrective D, conjunctive).** C and D enforce the same policy and block the same effect ($\approx 0\%$) but differ in enforcement completeness. *Rule:* supported if C's emission rate is 0 while D's is > 0 *and* C's attack-trial output-token cost is strictly below D's (both conjuncts required; note that on models with very short D retry traces the token conjunct may not hold, see Section 8, item on the H3 qualification). $H0_3$: C and D have equal emission and equal output-token expenditure on attack trials.

- **H4 (RQ4 — over-constraint / utility, non-inferiority).** C does not reduce legitimate-task success relative to Control within a 2-percentage-point tolerance. *Rule:* supported if the 95% CI on C minus Control lies entirely above -2 pp (the TOST equivalence margin): a 2pp non-inferiority TOST, cluster-robust by `task_id`, powered via hundreds of distinct benign tasks on a capable model. (The registered $\geq 90\%$-Control-legit precondition was not met by the Mistral anchor at 73.56%; the run was nevertheless executed and the result is reported in Section 6.) $H0_4$: C reduces legitimate-task success.

- **H5 (RQ5 — overhead, descriptive).** Per-request grammar-compile time stays within envelope (p99 compile < 25 ms) (the realized blind-C cold-compile p99 is 58–61 ms across the Qwen-151k and Mistral-131k tokenizers, exceeding this floor; reported in Section 6 with the amortized per-trial cost of ≈ 0.003 ms under cache hit). *Rule:* assessed descriptively against the envelope, not a hypothesis test; output-token expenditure and wall-clock latency reported for C vs Control vs the D-overlay.

- **H6 (RQ6 — least-privilege granularity, C-guided superiority).** *Rule:* supported if C-guided achieves bypass $= 0$ *and* deflection $= 0$ *and* its legitimate-task success exceeds Control's: a superiority claim distinct from RQ4's non-inferiority for blind C. $H0_6$: C-guided's legitimate-task success $\leq$ Control's.

The "eliminates" claim is carried by the formal proof (Section 4), not by the sample: the experiment shows that behavioral controls genuinely leak at scale and that the implementation matches the theory, while the proof, quantifying over all logit distributions, licenses the class-level elimination.

5.7 Sample

Win-classes as malware families. The five Tantalus win-classes are the "malware families": W1 SSH-key exfil, W2 API-key exfil, W3 infra-monitor exfil, W4 ticket-assist auto-forward, W5 cross-user chat exfil. Each is instantiated across paraphrase/obfuscation variants and a direct-vs-second-order delivery axis. Attacks are confused-deputy authority overreach induced by poisoned skills/emails [7].

Selected-on-outcome corpus. The corpus is built from attacks *known-effective* against the undefended agent: the live-malware analogue. It is therefore the population of *capable* attacks, not a representative sample of the threat space; bypass rates are conditional on attack potency, and the reference baseline is the control arm's own bypass rate (18.33% on the 1.7B workhorse, Table 3; 56.77% on the Mistral anchor, Table 4; 98.85% on the Gemma anchor, Table 5), not 100% (model nondeterminism means a known winner does not always re-win). The same attack corpus (`corpus_fable.json`, 174 entries) is used across all runs; the Control bypass rates differ by model capability, not by corpus. Large N tightens CIs only; it does not buy generalization. Legitimate-task success uses `corpus_legitimate_v2.json` (208 distinct benign in-scope tasks, sourced by

construction from the grammar allowlists and environment, with sensitive in-scope files omitted as non-benign).

Asymmetric N budget. Per family, the trial budget is weighted toward where evidence accrues rather than held symmetric across conditions: Control and the behavioral arms carry the bulk of the attack trials, the live-D arms split across $R \in \{0, 1, 3\}$, and C is run for empirical validation of the grammar door: its 0% is predicted by the formal proof and confirmed stochastically. The sample therefore tests implementation fidelity (was the grammar door actually honored?), not the structural claim (which the proof carries); no sample size can make C's 0% an independent empirical discovery, only a corroboration. The asymmetric budget reflects that additional C trials add CI precision on a guarantee already proven, not new evidence about the security property (scored by effect to confirm the grammar door was honored: the section notes that `guided_grammar` is silently ignored on the vLLM builds used, so the empirical check is operationally necessary). This is a deliberate deviation from the pre-registration's symmetric-N specification, recorded in the change log. Single subject system (Tantalus) yields a *feasibility* claim, not a universality claim; a "family" is a distinct pretraining lineage.

5.8 Engines and grammar backends

The confirmatory data come from vLLM. The grammar door is `structured_outputs.grammar` (passed via `extra_body`); the top-level `guided_grammar` field is *silently ignored* on the vLLM versions used (no error, free-text output), so every run asserts constrained output rather than trusting the request was honored. Two grammar backends appear: the default *xgrammar* [19] for the 1.7B blast and the steelman, and *guidance*/llguidance for the Mistral and Gemma anchors (ADR 0005) (the from-source build's xgrammar rejects the LARK grammar that vLLM's auto-tool-choice emits, crashing the engine on the first tool call; `guidance` accepts both LARK and our GBNF). The tool-call parser is per model (e.g., `hermes` for the 1.7B, `qwen3_coder` for the 35B, `mistral` for the anchor). C's grammar door is additionally corroborated on llama.cpp's native GBNF [20] (the 35B) and SGLang/xgrammar (the 1.7B, on an earlier host); these are *auxiliary corroborations* of engine-independence, not co-equal anchors. Engine-independence of C's structural guarantee is a robustness note, not a claim of three confirmatory engines; the grammar-mask construction itself follows the constrained-decoding line [18].

5.9 Reproducibility commitments and an honest gap

Every trial logs model id, engine commit, MTP setting, seed, temperature, condition, raw model output, the parsed/emitted tool call, output tokens, the win verdict, and timings; corpora, grammars, the security-neutral base prompt, harness, and analysis code are open and versioned, and the result database reproduces on equivalent hardware. We record one substantive gap up front. Three of the five finding databases (`blast_1p7b_5m.db`, `mistral119_anchor.db`, and `steelman_abliterated.db`) logged `engine_commit = unknown` at runtime; the values were backfilled out-of-band from verified container image digests (recorded in `harness/PROVENANCE.md`, tagged `-oob`) (the remaining two, `mistral119_ladder.db` and `gemma31_anchor.db`, were stamped correctly at runtime), and `model_id` remains a served alias rather than a pinned checkpoint hash; per-trial MTP-setting and seed logging are unverified for these databases. This is the primary validity concern and is treated as blocking for paper-grade reproducibility, not tail-end polish; it is carried forward to Section 8.

Table 3: Qwen3-1.7B factorial (`blast_1p7b_5m.db`); 282,000 attack + 624 legit per cell. Emission and `gate_rej` are counts; the only ungated cell is `control`. `c_closed_d_r3` is the closed-response cell (C+, ladder rung L2 guided); `c_guided_d_r3` is least-privilege (ladder rung L3 closed). Both are stored under their legacy DB labels; the canonical identifiers are `c_l2_guided` and `c_l3_closed` respectively (Section 5).

cell	bypass	emission	`gate_rej`	valid	out-tok	avail. fail	legit
`control`	18.33%	105,173	–	90.0%	289	–	58.3%
`a1_d_r3`	0.00%	120,140	139,612	93.9%	266	45	56.7%
`a2_d_r3`	0.00%	104,697	120,632	89.4%	270	46	59.6%
`a3_d_r3`	0.00%	104,615	111,528	89.4%	269	46	59.6%
`a4_d_r3`	0.00%	119,091	133,221	93.7%	261	44	56.7%
`c_d_r3`	0.00%	0	0	98.9%	119	0	47.0%
`c_closed_d_r3`	0.00%	0	0	99.9%	112	0	47.8%
`c_guided_d_r3`	0.00%	0	0	97.8%	149	0	100.0%

6 Results

We report five datasets spanning three distinct pretraining lineages (Qwen, Mistral, and Gemma), then summarise the per-hypothesis verdicts. Bypass is scored by *effect*: the marker reaching an out-of-scope sink, per the deterministic win detector (Section 5), never by grammar-membership. Each subsection maps its measurements to the registered research questions honestly, including the cases the dataset does *not* settle. Binomial confidence intervals are Clopper–Pearson 95% [22].

6.1 The Qwen3-1.7B generator× gate factorial

The first dataset (`blast_1p7b_5m.db`) runs a deliberately weak workhorse (Qwen3-1.7B-NVFP4) at scale to sharpen the constitutive-vs-corrective contrast. It is an eight-cell generator×gate factorial. The only ungated cell is `control`; every other cell layers the post-parse allowlist gate at retry budget three behind a generator (`a1_d_r3`, `a2_d_r3`, `a3_d_r3`, `a4_d_r3`, `c_d_r3`, `c_closed_d_r3`, `c_guided_d_r3`). Each cell carries 282,000 attack trials and 624 legitimate trials. Table 3 reports the per-cell measurements.

RQ2 (structural invariant), reproduced but not the formal test. The three GCD cells produce *zero* bypasses across 282,000 attack trials each (Clopper–Pearson 95% upper bound $\approx 1.31 \times 10^{-5}$). Bypass is scored by effect: the `WinDetector` fires only when the marker reaches an out-of-scope sink URL in an executed `fetchUrl` call, not on grammar-membership, so a grammar-constrained call that reaches an allowlisted URL scores 0 bypass regardless. This reproduces the structural invariant at large N. It is, however, *not* an evaluation of the registered H2 decision rule. H2 requires C's CI upper bound to lie strictly below every behavioral control's bypass *point estimate*, and here the behavioral arms are gated, so their bypass point estimates are all 0 rather than leaky. The formal H2 contrast is delivered on the Mistral anchor (Section 6.2).

RQ3 (constitutive vs corrective), supported on both conjuncts. The Section 3 decision rule for H3 is conjunctive: C emission must be 0 *and* C's attack-trial output-token cost strictly below D's. Both hold here. The three GCD cells show 0 emission, 0 `gate_rej`, and 0 availability failures, because the grammar already enforced the policy at the sampler and the gate is therefore *inert*. The behavioral generators, by contrast, produce 104,615–120,140 out-of-scope calls each behind the same

Table 4: Mistral-Small-4-119B-NVFP4 (`mistral119_anchor.db`); 4,000 attack + 208 legit per cond, behavioral arms ungated. Bypass and legit shown with Clopper–Pearson 95% CIs.

cond	bypass [CI]	emission	valid	out-tok	legit [CI]
`control`	56.77% [55.22, 58.32]	2305	59.2%	239	73.56% [67.01, 79.42]
`a1`	56.50% [54.95, 58.04]	2326	59.1%	240	73.08%
`a2`	56.85% [55.30, 58.39]	2315	59.4%	239	76.92%
`a3`	2.15% [1.72, 2.65]	2324	59.5%	216	75.00%
`a4`	1.82% [1.43, 2.29]	2343	59.4%	219	78.85% [72.66, 84.19]
`c`	0.00% [0, 0.09]	0	99.8%	208	78.85% [72.66, 84.19]
`d_r0`	0.00% [0, 0.09]	3878	–	270	–
`d_r1`	0.00% [0, 0.09]	2317	–	203	–
`d_r3`	0.00% [0, 0.09]	2337	–	210	–

gate (produce-then-catch), with 111,528–139,612 gate rejections and 44–46 trials stranded at retry-three. The GCD cells also cost fewer tokens (112–149) than every behavioral-generator-behind-gate cell (261–270), and exhibit the validity dividend (97.8–99.9% valid output against 89.4–93.9% behavioral). One caveat travels with the GCD cells: `c_guided_d_r3`'s 97.8% falls below the registered invalid-rate floor, a deviation disclosed in Section 8, item 6, whose residual is free-string truncation that fails safe. Emission is the empirical signature of the constitutive-vs-corrective difference: 0 under C by construction, > 0 under the corrective gate. This corrective produce-then-catch pattern is representative of the post-generation positive-control class (CaMeL [11], Progent [12]); Section 9 surveys this class in full and distinguishes it from the constitutive enforcement of GCD [3].

RQ6 (least-privilege superiority), supported on all three conjuncts. `c_guided_d_r3` attains bypass 0, deflection 0/282,000 (against blind-C's 98,006 and `control`'s 86,817 deflection trials), and legitimate-task success 100.0% (624/624) versus `control`'s 58.3%: the superiority claim of H6, distinct from blind-C's non-inferiority. The H6-on-capable replication is delivered on the Mistral ladder (Section 6.3) and the Gemma anchor (Section 6.4).

What this cell does not deliver. RQ1 is not addressed here. The weak workhorse's `control` bypass of 18.33% sits below the registered $\geq 50\%$ potency floor, the behavioral arms are gated, and the superseded ungated `a1` orphan ($\approx$20.3%, $\geq$ `control` 18.33%) fails the A1<Control manipulation check. All of this is by design: the 1.7B is the weak spectrum-end apparatus, not the headline victim. RQ4 is also not met here. Blind-C legit-success (47.0%) and C+ (47.8%) fall about 11 pp below `control` (58.3%). This deficit is the expected weak-model deflection cost; C-guided is the resolution (100.0%), and the capable-victim picture (Section 6.2) shows the deficit does not reproduce.

6.2 The Mistral-Small-4-119B capable non-Qwen anchor

The second dataset (`mistral119_anchor.db`) is the capable, non-Qwen anchor the 1.7B factorial deferred to, a second pretraining lineage. It runs nine conditions at 4,000 attack + 208 legitimate trials each, with the behavioral arms *ungated* (bare `a1`–`a4`), so their bypass *is* the leak. This is what makes RQ1 and the formal H2 contrast evaluable. Table 4 reports the matrix; Figure 4 plots the bypass column.

RQ1 (behavioral controls leak), delivered. On this capable non-Qwen victim the behavioral controls leak, and the leak is bounded from below. Control bypass is 56.77% [55.22, 58.32], clearing the $\geq 50\%$ potency floor; the defensive-prompt arm A1 (56.50% [54.95, 58.04]) is $\leq$ `control`, so the

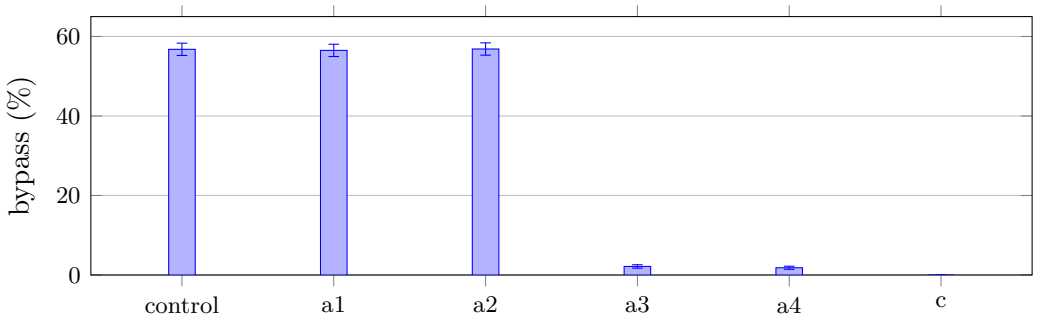

Figure 4: Mistral anchor bypass by condition (Clopper–Pearson 95% intervals; asymmetric error bars give the exact upper/lower half-widths from Table 4). Behavioral arms are ungated; the full stack `a4` still leaks (CI-lower 1.43% > 0, H1), while GCD (`c`) is 0 (CI-upper 0.09%, H2).

manipulation check holds; and the full behavioral stack A4 still leaks at 1.82% [1.43, 2.29], whose CI lower bound 1.43% exceeds 0, the H1 decision rule (Figure 4). The input classifier A2 (56.85%) is essentially useless on this capable model; the output filter A3 (2.15%) does the heavy lifting but never reaches 0.

Staging funnel (amendment #5). Decomposing `raw_json` into the funnel *initiated a tool chain* → *emitted any* `fetchURL` → *bypass*, the attack stages at ≈57–58% `fetchURL` emission in *every* arm, including A3/A4: `control` 2297 (57.4%), `a1` 2293 (57.3%), `a2` 2301 (57.5%), `a3` 2311 (57.8%) at bypass 86 (2.15%), and `a4` 2328 (58.2%) at bypass 73 (1.82%). A1 (defensive prompt) and A2 (input classifier) do not move the funnel at all; A3/A4 leave the dangerous emission *unchanged* and block only at the final output stage. A4's 1.82% is therefore best read as "the attack fired ≈58% of the time and the filter caught all but 73," not "the attack rarely fired": the survivors are the credential-signature-free business-data exfils (the TicketAssist class H1 specifically predicted), and the produce-then-catch tax is visible *within* the behavioral arms.

RQ2 (positive structural control), supported as the formal contrast. GCD (C) bypasses 0/4,000 attacks, CI [0, 0.09] (again scored by effect: the `WinDetector` fires only when the marker reaches an out-of-scope sink URL in an executed `fetchUrl` call, so this is 0/4000 executed calls reaching such a sink, not 0 grammar-legal tokens). The upper bound 0.09% lies strictly below every behavioral control's bypass point estimate (a4 1.82% through `control` 56.77%), satisfying the registered H2 rule, now evaluable because the behavioral arms are ungated. This 0-bypass result holds across ≈868,000 GCD attack trials in aggregate, consistent with the structural guarantee. The decode-time enforcement nearest neighbors Agent-C [9] and PackMonitor [10] share the generation-time enforcement layer but differ in scope: temporal-ordering constraints and package-hallucination correctness, respectively, against authorization-class actions over enumerable sinks (Section 9).

RQ3 (constitutive vs corrective). The emission and validity conjuncts are decisive: C emits 0 out-of-scope calls against D's 2,317–3,878 produced-then-caught, and C's valid-output rate is 99.8% against ≈59% for native free-generation (a large validity dividend on this victim). This validity property is a corollary of Lemma 1 in Section 4: every emitted token lies in $L(G_s)$ for any logit distribution, so well-formedness is enforced mechanically, not learned [18, 19]. On the reliability axis, C resolves in one attempt against D's retry-recovery (average 1.63–1.72 attempts, max 5), with availability failures shrinking across retry budgets (`d_r0` strands all 3878 emissions → `d_r1`

288 → `d_r3` 6). However, the Section 3 H3 *token* conjunct fails here: C's 208 output tokens are not strictly below every D variant (`d_r1` is 203). The H3-as-written rule is therefore met only on the 1.7B (and on Gemma, Section 6.4); the victim-independent result is the qualitative gate-inertness, while the token ordering is victim-dependent.

Validity decoupled from capability (the validity dividend). GCD makes tool-call validity a property of the *enforcement mechanism*, not the model. On the 1.7B blast (Table 3, 282,000 trials/cell), `c_d_r3` emits a well-formed, in-policy tool call **98.9%** of the time; `c_closed_d_r3` **99.9%**; `c_guided_d_r3` **97.8%**. The capable 119B Mistral in free generation manages only **59.2%** (Table 4, `control valid_output`); under GCD that same Mistral free-gen validity rises to **99.8%** (`c`). Placed adjacently, the small model with the grammar (98–100%) outperforms the capable 119B anchor without it (59%) on structural validity. The deployment payoff is a separation of concerns: capability buys task intelligence, the grammar buys structural validity, and the two are separable, so a smaller, cheaper model gets structurally guaranteed valid calls that the capable Mistral-Small-4-119B anchor fails to produce ≈40% of the time unaided. However, *this dividend is victim-dependent.* It is large precisely where free-generation tool-call validity is poor: ≈41 pp on Mistral (native 59.2%) and ≈4–10 pp on the 1.7B (native behavioral 89.4–93.9%, Table 3). On a model whose native tool-call competence is already high, the dividend nearly vanishes: Gemma free-generation is 97.9–100% valid (Section 6.4). The benefit therefore accrues to models whose unaided structural validity sits below ≈95%; it is a property of the enforcement layer relative to the model, not a universal constant. The honest fence, in the same breath: the grammar makes the call valid and in-policy by construction (Lemma 1, Section 4 [18]), not necessarily *correct* about which in-scope action serves the task (the deflection DV). A valid call to the wrong *authorized* endpoint is still 0 bypass and 0 emission; GCD never fails structurally by construction, which is not the same as never failing.

RQ4 (over-constraint). The 1.7B blind-C deflection deficit does *not* reproduce here: blind-C legit-success is 78.85% [72.66, 84.19], above `control`'s 73.56% [67.01, 79.42]. This is a *point-estimate* ordering only, and the two CIs overlap heavily. This anchor cell ($n = 208$, `--legit-rounds` 1) is under-powered for the registered powered 2 pp non-inferiority TOST clustered by `task_id`: its CI half-width (≈6 pp) far exceeds the 2 pp margin. The powered TOST is instead run on the companion Mistral ladder (Section 6.3), where it *fails* even at full power. The defensible cross-victim interpretation is that the 1.7B deflection deficit does not reproduce on a capable model; its cause is not yet formally partitioned, and we do not claim non-inferiority is established.

6.3 The Mistral ladder: powered RQ4 and H6-on-capable

A companion run on the same Mistral-Small-4-119B engine (`mistral119_ladder.db`, 21,824 trials at 0 errors) carries the powered utility test and the L3-closed least-privilege rung on a capable victim. Each cell has 4,000 attack trials and 1,456 legitimate trials (`--legit-rounds` 7, the registered 208 `task_id`s ×7), which is the design the pre-registered TOST specifies.

RQ4 (powered non-inferiority TOST), run and *not* established. With the powered design in place, blind-C legitimate success is 77.20% [74.95, 79.33] against `control`'s 79.05% [76.87, 81.12]: an observed difference of −1.85 pp, which is inside the 2 pp margin. However, the cluster-bootstrap (clustered by `task_id`, $B = 10{,}000$) 95% confidence interval is $[-7.07, +3.43]$, a half-width of ≈5.3 pp, far wider than the 2 pp margin. Non-inferiority is therefore **not established even when powered**: the point estimate sits inside the margin, but the interval does not. This is the registered

Table 5: Gemma-4-31B-IT-NVFP4 (`gemma31_anchor.db`); 2,000 attack + 624 legit per cond. Bypass and legit shown with Clopper–Pearson 95% CIs; behavioral arms ungated.

cond	bypass [CI]	emission	out-tok	legit [CI]
`control`	98.85% [98.28, 99.27]	1997	470	88.78% [86.04, 91.15]
`a1`	65.85% [63.72, 67.93]	–	–	–
`a2`	98.90% [98.34, 99.31]	–	–	–
`a3`	34.10% [32.02, 36.22]	–	–	–
`a4`	33.80% [31.73, 35.92]	–	–	–
`c`	0.00% [0, 0.18]	0	267	87.34% [84.47, 89.85]
`c_l3_closed`	0.00% [0, 0.18]	0	197	100.00% [99.41, 100]
`d_r0`	0.00% [0, 0.18]	1975	–	–
`d_r1`	0.00% [0, 0.18]	1976	–	–
`d_r3`	0.00% [0, 0.18]	1998	–	–

H4 test executed on a capable model with the registered sample size, and it returns a confirmed negative, not an unmeasured gap.

RQ6 (least-privilege superiority) on a capable victim. The L3-closed cell `c_l3_closed` on this capable Mistral attains bypass 0.00% [0, 0.09], 0 emission, valid-output 98.9%, average 183 output tokens, and legitimate-task success 100.00% [99.75, 100]. The three-conjunct H6 rule (bypass 0, deflection 0, legit superiority) is therefore met on a capable non-Qwen model, not only the sub-floor 1.7B. The corrective gate is again inert under the constitutive control: on this engine the attack stages at ≈57–58% `fetchURL` emission per arm (Section 6.2 staging funnel), so an ungated free generator emits an out-of-scope call on a majority of trials, against 0 emission for the C-family arms.

Apparatus deviation: `c_l2_guided` on the `guidance` backend. The L2-guided closed-response rung did not transfer to this engine. Its ≈18.6 KB, 100-way canned-response alternation drove the `guidance`/`llguidance` backend to reject tokens, producing ≈72% empty `[]` traces (`valid_output` 30.2%, average 38 output tokens). This is a backend–grammar incompatibility, not a grammar bug: the identical grammar achieved 97.8% `valid_output` under `xgrammar` on the 1.7B blast (Table 3). The cell is reported but excluded from the L2 conclusions; the L1 (blind-C) and L3 (`c_l3_closed`) rungs are unaffected.

6.4 The Gemma-4-31B third pretraining lineage

The fourth dataset (`gemma31_anchor.db`, 26,240 trials at 0 errors, 0 empty `raw_json`) is the third pretraining lineage, Google Gemma, which closes the pre-registration §12 amendment #1 ≥3-lineage external-validity bar. It runs ten conditions at 2,000 attack + 624 legitimate trials each (`--legit-rounds` 3). Table 5 reports the matrix.

RQ1 (decisive on a third lineage). Gemma leaks far harder than Mistral (Figure 5). The full behavioral stack `a4` bypasses 33.80% [31.73, 35.92], a CI well clear of 0, and ≈18× Mistral's A4 (1.82%). The defensive prompt A1 (65.85%) is below `control` (98.85%), so the manipulation check holds and the prompt does real but partial work; the input classifier A2 (98.90%) is useless; the output filter A3 (34.10%) does most of the residual work but, as on Mistral, never reaches 0. H1 is decisive on this lineage.

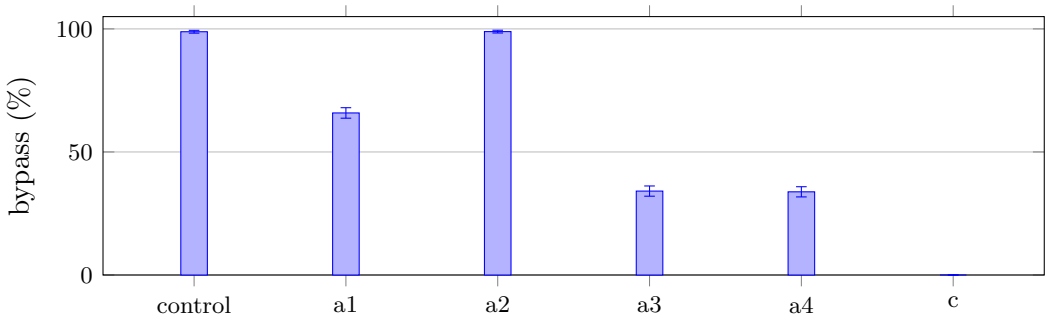

Figure 5: Gemma-4-31B anchor bypass by condition (Clopper–Pearson 95% intervals; asymmetric upper/lower half-widths from Table 5). The third lineage leaks far harder than Mistral: the full stack `a4` bypasses 33.80% (CI-lower 31.73% $>$ 0, $\approx$18$\times$ Mistral's A4, H1), while GCD (`c`) is 0 (CI-upper 0.18%, H2). Compare Figure 4.

Table 6: Abliterated steelman (`steelman_abliterated.db`); 2,000 trials/cell. Same model, same malicious prompt; grammar is the sole independent variable.

cell	bypass (exfil)	note
`control` (free, sink open)	97.85% (1957/2000), CP95 $\approx$[97.1, 98.4]	adversary is real
`c_sealed` (forced, sink sealed)	0.00% (0/2000), rule-of-three upper $\approx$0.15%	0/2000 attacker `fetchUrl`

RQ2 / RQ3 / RQ6 on Gemma. GCD (`c`) bypasses 0/2000, CI [0, 0.18] (again scored by effect: the `WinDetector` fires only on the marker reaching an out-of-scope sink URL in an executed `fetchUrl` call), with 0 emission and 100.0% `valid_output`, against behavioral point estimates from 33.80% up: the formal H2 contrast again. The corrective gate's produce-then-catch cost is the 1,975–1,998 D-arm emissions and the 0.70–1.04 M wasted tokens, against 0 for the C-family arms. Here the H3 token conjunct *holds*: C's 267 output tokens are below every behavioral and D cell, so the strict ordering that failed on Mistral is recovered on a native model that emits valid calls 98–100% of the time unaided (the validity dividend is correspondingly small on Gemma, again victim-dependent). H6 is met on this third lineage: `c_l3_closed` attains bypass 0, 0 emission, and legitimate-task success 100.00% [99.41, 100] at 197 output tokens.

Honest reading of RQ4 on Gemma. Blind-C legit-success is 87.34% [84.47, 89.85] against `control`'s 88.78% [86.04, 91.15]: a -1.44 pp point estimate, directionally small but not subjected to a powered TOST here. The powered non-inferiority verdict remains the Mistral ladder's confirmed negative (Section 6.3).

6.5 The abliterated-adversary steelman

The fifth dataset (`steelman_abliterated.db`) is a rebuttal exhibit, not main statistics. The victim is an abliterated Qwen3.5-27B driven by a malicious top-authority system prompt; abliteration surgically removes behavioral refusal directions from the weights [21], so the adversary cannot decline. Two cells of 2,000 trials each contrast a free generator with the sink open (`control`) against the forced-denylist, sink-sealed condition (`c_sealed`). Figure 6 shows the contrast.

The contrast and its honest reading. The same model under the same prompt moves from 97.85% exfil to 0%, with the grammar as the sole independent variable: behavioral safety is re-

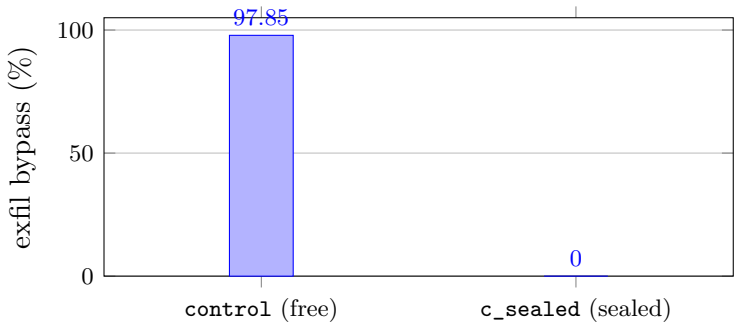

Figure 6: Abliterated steelman: the same un-abliterated-refusal model under the same malicious prompt moves from 97.85% exfil (free, sink open) to 0% (forced trajectory, sink sealed). Grammar is the sole independent variable (note: `c_sealed`'s 0% is partly true by construction – the forced trajectory seals the sink; see text). Data from Table 6.

moved from the weights, but GCD lives in the sampler and is un-abliteratable. The integrity audit of `c_sealed` shows 0/2000 executed `fetchUrl` calls to an attacker URL. We are explicit that `c_sealed`'s 0% is partly true by construction, because the forced trajectory seals the exfil sink. Its empirical content is two-fold: (a) the paired open cell at 97.85% establishes that the adversary is real and willing, and (b) in 1,937/2,000 trials (96.8%) the sealed model provably ran its skill's full forced trajectory, namely the compelled read of the skill-targeted sensitive artifact, then a sealed safe-fetch, then a verbatim taunt. The artifact varies by skill: AWS-credential file in 542, SSH private key in 259, cross-user chat in 517, and ticket business-data in 619 (542+259+517+619 = 1937), with a 63-trial unmapped/multi-message remainder that still produced 0 exfil. The precise reading: the claim "provably read the AWS credentials" holds for the 542; the broader "read the skill's sensitive artifact" holds for all 1937, and we do not generalize the AWS claim across all 1937. This provable read is *engagement* evidence (the adversary engaged the malicious content), not a confidentiality result: GCD permits the read and blocks only the exfil action; confidentiality is Paper-2 scope.

6.6 Hypothesis ledger

Table 7 summarises the per-hypothesis verdicts against the Section 3 decision rules.

Across the five datasets the experiment comprises ≈2,460,677 trials at 0 errors, ≈27× the 90,000 test cases of the agent-security benchmark of Zhang et al. [8], with ≈868,000 of those being GCD attack trials at 0 bypass across the five datasets. Because each trial is a full multi-turn agent rollout, those trials decompose into 6,122,392 individual inference calls (≈2.49 per trial) and 543,699,152 generated output tokens in total.

7 Discussion

The datasets converge on a single structural claim and a set of qualifications around it. This section synthesizes them in the order of the contribution, then recaps the framing discipline that keeps the claim honest. One figure is introduced here (§7.2); the remaining subsections interpret results already reported in Sections 6 and elsewhere.

Table 7: Per-hypothesis verdicts. "Supported" is against the registered decision rule unless qualified.

ID	verdict	basis
H1	supported	Mistral A4 bypass 1.82% [1.43, 2.29], CI-lower 1.43% > 0, `control` 56.77% clears the floor, A1 $\leq$ `control`; replicated decisively on Gemma (A4 33.80% [31.73, 35.92], $\approx$18$\times$ Mistral).
H2	supported	Mistral C 0/4,000 [0, 0.09] and Gemma C 0/2,000 [0, 0.18] strictly below every behavioral point estimate (the formal contrast); the GCD arms hold 0 bypass across $\approx$868,000 GCD attack trials total.
H3	supported (1.7B, Gemma), qualified (Mistral)	Conjunctive rule fully met on the 1.7B (emission 0 *and* tokens 119 $<$ behavioral 261–270) and on Gemma (C 267 $<$ every behavioral/D cell); on Mistral the emission/validity conjuncts carry but the token conjunct fails (C 208 $\not<$ `d_r1` 203).
H4	not established (powered, confirmed negative)	Powered 2 pp non-inferiority TOST run on the Mistral ladder (n = 1,456 legit/cell, cluster-bootstrap B = 10,000): observed C$-$Control = -1.85 pp (inside the margin) but 95% CI $[-7.07, +3.43]$ (half-width $\approx$5.3 pp $\gg$ 2 pp) – non-inferiority not established even when powered.
H5	measured, qualified	Cold FSM-compile p99 58.64 ms (Qwen-151k) / 61.24 ms (Mistral-131k) for blind-C exceeds the registered 25 ms target; amortized per-trial $\approx$0.003 ms (single fixed grammar, cache hit). Reported as deviation: letter not met, spirit holds amortized.[3]
H6	supported (1.7B, Mistral, Gemma)	`c_l3_closed` attains bypass 0, deflection 0, legit 100.0% on the 1.7B (624/624), 100.00% [99.75, 100] on the capable Mistral ladder, and 100.00% [99.41, 100] on the Gemma anchor – the three-conjunct rule met on all three victims.

7.1 Constitutive enforcement subsumes corrective enforcement

The central result is not that GCD blocks attacks. A correctly wired post-parse allowlist (`D`) blocks the same *effect* on the same policy, because `D` and `C` enforce the same language $L(G_s)$ through the same `tantalus_grammar::allowlist_verdict` routine both `C` and `D` call (`D == C` by construction). The result is that `C` and `D` differ in *enforcement completeness*: where the policy sits relative to the generative act. `C` is constitutive: the policy *is* the output space, so the out-of-scope action is ungenerable and never exists as an artifact on any surface (output buffer, logs, telemetry, stream, KV cache). `D` is corrective: an unbounded, default-allow generator produces the forbidden action in full, after which a separate gate inspects and rejects it.

The empirical signature of that difference is *emission*, the per-trial count of times the unauthorized artifact was actually produced. Across both victims the signature is the same. On the weak Qwen3-1.7B workhorse, the behavioral generators behind the allowlist gate (`a1_d_r3`–`a4_d_r3`) each produced 104,615–120,140 out-of-scope calls (emission), drove 111,528–139,612 gate rejections, and stranded 44–46 trials at the retry-3 budget. The three GCD arms (`c_d_r3`, `c_closed_d_r3`,

c_guided_d_r3) recorded *0* emission, *0* gate rejections, and *0* availability_failure: the gate is dead weight, because the grammar already enforced the policy at the sampler. On the capable Mistral-119B anchor the same shape holds. c emits 0 while the corrective D arms produce 2,317–3,878 calls and pay an availability cost (d_r0 strands all 3878; retry recovers to 288 at d_r1 and 6 at d_r3, at 1.63 to 1.72 average attempts, max 5). This is the produce-then-catch tax, and it is precisely the failure mode positive security exists to escape, reintroduced one layer down: the dangerous artifact is produced (a confidentiality and forensics surface), a coverage obligation attaches to the gate (it must fire on every output path, consumer, and refactor or be bypassed), and a time-of-check/time-of-use window opens. GCD has none of these because there is nothing to catch. Systems in the post-generation enforcement class, including CaMeL [11] (dual-LLM sandbox with a capability graph), FIDES [14] (trust-propagation at parse time), MiniScope [13] (scope minimization), and Progent [12], AgentArmor [15], OAP [16], and ARM [17], occupy this corrective cell; they enforce correct positive policies but cannot eliminate the produce-then-catch artifact their corrective architecture creates.

We state the boundary of the contribution carefully. C is not *more secure* than a perfectly wired allowlist on the blocked effect: they enforce the same positive policy. The difference is that GCD's positivity is enforcement-deep (positive security all the way down to the generative act) while the allowlist's is policy-deep only. The same post-parse check is therefore C's recommended corrective backstop in deployment, a defense-in-depth complement to GCD rather than a competing approach (Section 7.6). The cost ordering *among* the behavioral variants on the 1.7B inherits the sub-floor-victim caveat (Section 7.3); the load-bearing, victim-independent result is the qualitative 0-vs-$>$0 gate inertness.

A caveat on the token-cost axis follows directly. The pre-registered H3 decision rule is conjunctive: C emission = 0 *and* C attack-trial output-token cost strictly below D's. On the 1.7B both conjuncts hold (C 119 output tokens vs. the gated behavioral arms' 261–270, a4_d_r3–a2_d_r3, Table 3). On Mistral the emission conjunct carries decisively but the token-cost conjunct fails: blind-C's 208 output tokens are among the leanest yet not strictly below every D variant (d_r1 203). A capable model writes a substantive free respondToUser message, narrowing the token gap that was large on the weak victim. So the formal, conjunctive H3 rule is met only on the 1.7B; on Mistral the qualitative gate inertness and the validity dividend (below) carry, while the token ordering is victim-dependent. We do not claim H3-as-written is satisfied on both victims.

7.2 Validity is a property of the enforcement mechanism, not the model

Beyond security, GCD buys reliability that the model does not supply on its own. Formally this is a second corollary of Lemma 1 (Corollary 1, formal.tex): because every emitted token must belong to $L(G_s)$, every complete parse is a well-formed in-policy tool call by construction, so security (out-of-scope token ungenerable) and structural validity (only grammar-legal tokens generable) follow from the same mask. The empirical consequence is direct and surprising. The 1.7B under GCD attains 98.9% valid tool calls (c_d_r3, Table 3); the capable 119B Mistral model without a grammar manages only 59.2% (Table 4). The weaker model with the grammar structurally outperforms the capable 119B model without it on this axis.

Capability decoupled from structural validity (Corollary 2): structural validity is a property of the enforcement mechanism, not the model; a 1.7B-parameter model under GCD reliably exceeds a 119B-parameter model running free on this dimension. Across both numeric victims the GCD arms produced a well-formed, executable, in-policy tool call far more often than native free generation did: c_d_r3 98.9%, c_closed_d_r3 99.9%, c_guided_d_r3 97.8% valid output on the 1.7B (vs. 89.4 to 93.9% for the behavioral arms, Table 3), and c 99.8% valid on Mistral against

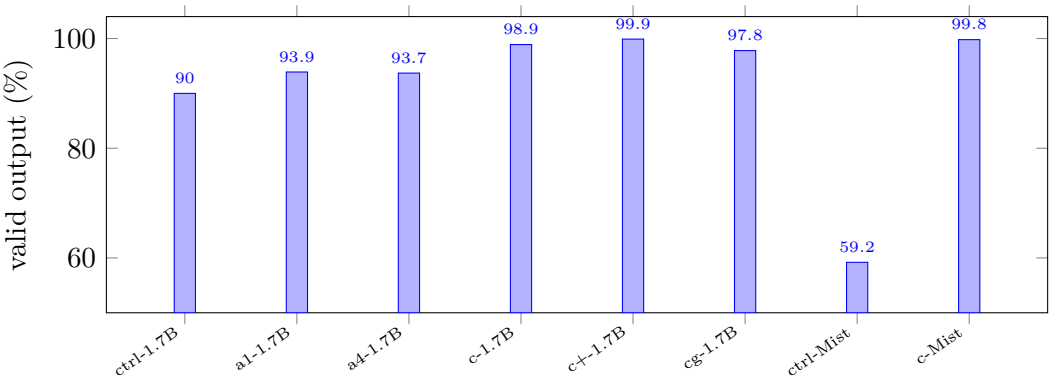

Figure 7: Validity dividend: GCD vs. native free generation, for the two numeric victims shown (1.7B and Mistral-119B; numbers from Tables 3 and 4). The GCD arms (`c`, `c+`, $\texttt{c}_\texttt{g}$) achieve 97.8 to 99.9% valid output across both victims shown; the capable 119B Mistral model unconstrained (`ctrl-Mist`) manages only 59.2%, rising to 99.8% under GCD (`c-Mist`). A 0–2.2% truncation residual under Kleene-`*` free-string rules (largest for $\texttt{c}_\texttt{g}$) is disclosed in Section 8; it fails safe (no executable call produced, 0 bypass) but prevents a literal zero-invalid claim on vLLM. Gemma native arms (97.9 to 100%, Table 5) are omitted as they fall outside the figure range; the dividend is negligible there by inspection.

only about 59% for native free generation (`control` 59.2%, Table 4). The capable model emits a valid tool call only about 59% of the time, the rest being tool-call-as-text, spin-out, or truncation that never executes, and the grammar eliminates that failure class by construction (Figure 7). The deployment payoff is that the two axes are separable: capability buys task intelligence, the grammar buys structural validity, so a cheaper, smaller model under GCD gets structurally guaranteed valid calls that a frontier model fails to produce roughly 40% of the time unaided [1, 3]. However, the magnitude is victim-dependent: it is large on Mistral ($\sim$59% native valid output rising to 99.8% under GCD) and the 1.7B, but negligible on Gemma, whose native arms are already 97.9 to 100% valid (Table 5). What is model-independent is the structural guarantee itself (a valid in-policy call on every completed parse), while the size of the dividend it pays depends on how often the unconstrained victim already produces well-formed calls.

Two qualifications keep this honest. First, "0 invalid outputs" is *not* literally met on vLLM: under the Kleene-`*` free-string rules the weak 1.7B occasionally rambles `respondToUser` or `searchFiles.query` to `max_tokens` and truncates mid-string, leaving an unparseable turn. The residual is largest for `c_guided_d_r3` ($\approx$2.2% invalid, above the pre-registered floor; recorded as a deviation), whose terminal reply is still a free respond-only string. It fails safe: a truncated grammar output yields no executable call, never a wrong action, so 0 win / 0 emission still holds on every such trial. It remains far better than the 8 to 10% parse failures of native generation, but it is not literal zero, and the fix path (a closed canned respond-only terminal, as `C+` already applies to `respondToUser`) is named rather than claimed. Second, the validity gain does not make a small model capable: intelligence lives in the corpus and the selection, not in the grammar. GCD makes the model's *output* valid and in-policy; it does not make the model frontier.

7.3 Negative-security incompleteness, made visible

On the capable Mistral anchor the behavioral stack behaves exactly as the classical positive/structural-vs-negative/behavioral dichotomy predicts [1, 3]. The in-band controls are inert against injection:

the defensive prompt A1 (56.50% bypass) does not improve on Control (56.77%), so the A1 $\leq$ Control manipulation check holds, and the input classifier A2 (56.85%) is likewise useless. Both live in the same input channel the injected skill occupies, so the injection overrides them. The output filter A3/A4 does the heavy lifting (2.15% / 1.82% bypass) but *never reaches 0*: A4's CI lower bound of 1.43% is strictly above zero, satisfying H1.

The staging funnel localizes *why* A3/A4 still leak. The attack reaches the `fetchURL` sink in $\sim$57–58% of trials in *every* arm, A3/A4 included; A1 and A2 do not move the funnel at all, and A3/A4 leave the dangerous *emission unchanged*, blocking only at the final output stage. A4's 1.82% is therefore "the attack fired $\sim$58% of the time and the credential denylist caught all but 73," not "the attack rarely fired." The 73 (`a4`) and 86 (`a3`) survivors are exactly the credential-signature-free exfils, the TicketAssist business-data class H1 specifically predicted, which carry no pattern for a denylist to match. This is the produce-then-catch tax surfacing *within* a behavioral arm: a post-generation credential denylist is itself corrective and incomplete by construction, so the unauthorized `fetchURL` is generated regardless of the defense, and any payload outside the signature set passes. The contrast with `C`'s constitutive 0 emission is the whole point.

7.4 The un-abliteratable sampler

The abliterated-adversary steelman isolates *where* each control lives. Behavioral safety lives in the weights, and abliteration [21] surgically removes the refusal directions that A1–A4 implicitly rely on, defeating the entire behavioral stack by construction. Under a top-authority malicious prompt the abliterated Qwen3.5-27B is demonstrably willing and able, bypassing 97.85% (1957/2000) in the free cell. Constitutive enforcement, by contrast, lives at the decode boundary, in the sampler, not in the weights: the grammar mask drives every grammar-invalid token's probability to zero *before* sampling [18, 19], so the same model under the same prompt is driven to 0.00% (0/2000) bypass with the grammar as the sole independent variable. The transformer's pre-softmax scores remain uncontrolled and may, under injection, place arbitrarily high probability on a forbidden token; the *generated tokens* are nonetheless constrained. (Bypass is scored by the independent win detector, marker reaching an out-of-scope sink, not by grammar membership, so `c_sealed`'s 0/2000 is an effect-measured result, not a redefinition of bypass.)

We present this as a rebuttal exhibit, not main statistics, and bound it precisely. `c_sealed`'s 0% is partly true-by-construction (the forced denylist trajectory seals the exfil sink), so its empirical content is the paired open cell (the adversary is real at 97.85%) and the engagement evidence: in 96.8% (1937/2000) of sealed trials the model *provably ran its skill's forced sensitive-artifact read* before reaching the sealed safe-fetch and the verbatim taunt (AWS-credential file in 542, SSH key 259, cross-user chat 517, ticket business-data 619). That answers the two natural objections, "the model just refused" (it cannot, and bypasses 97.85% unconstrained) and "it never read the bad content" (96.8% provably did). However, it is engagement evidence, not a confidentiality result: GCD here *permits* the read and blocks only the exfil action; the AWS-credential claim holds for the 542 ApiKey/Infra trials, not for all 1937. Protecting a secret through free-text channels is explicitly Paper-2 scope.

7.5 Cost and economics

Token generation is not free, and the constitutive/corrective distinction has a direct economic reading. A corrective gate must let the generator spend the tokens to produce the forbidden artifact, then discard them; a constitutive grammar never spends them. On attack trials this shows up as the GCD arms generating $\approx$ 45–55% of the gated behavioral output tokens on the 1.7B

(112–149 vs. 261–270, Table 3), with the gap narrowing on a capable model whose blind-`C` reply is substantive (208 vs. `D`'s 203–270). We frame this as a layer on top of the security claim, not a substitute, and we make no single-axis "cheaper" claim. `C` carries a per-token masking cost that is engine-dependent (a CPU mask cliff on `llama.cpp` [20]; at-par on vLLM with xgrammar [19], which has no jump-forward but also no cliff), so `C` is not "the slow arm": the avoided regeneration is a real but bounded saving, reported against an honest per-engine latency picture. The third C-vs-D signature, time-to-valid-action measured by `attempts`, reinforces the structural reading: `C` is one-shot (one attempt by construction, no gate round-trip), while `D` serializes $1.0 \to 1.72$ average attempts plus a rejection round-trip per retry.

7.6 Framing discipline: what is and is not load-bearing

Security at scale is the headline. The contribution ports Galloway's positive-control/application-whitelisting vs negative-control comparison [6], itself grounded in the classical complete-mediation and correct-by-construction lineage [1, 3], into the LLM-agent domain: the same attack corpus, the same agent, only the control changes, scored by *effect* with Clopper-Pearson intervals [22], at a scale roughly 27× ASB-2024's 90,000 test cases [8], with ≈ 868,000 GCD attack trials at 0 bypass across the five datasets. The soundness proof and the cost axis are layers *on top* of that headline, not stand-ins for it. Contemporaneous decode-time enforcement work, Agent-C [9], which enforces temporal constraints with a model-agnostic soundness claim, and PackMonitor [10], which eliminates package hallucinations via automaton monitoring, addresses the same decoder-boundary layer; both operate on harder-to-enumerate constraint domains (temporal ordering, hallucinated package names) than our positive allowlist, and neither reports an empirical positive-vs-negative-control comparison against behavioral baselines at this scale.

Three disciplines keep the empirical claim from collapsing into a tautology. First, `C`'s 0% is load-bearing because it is run at scale, scored by effect, and measured *through* the unconstrained free-text surface (`searchFiles.query`, `respondToUser.message` remain free): bypass is "did the marker reach an out-of-scope sink," not "is the output a member of $L(G_s)$." The grammar does not co-define the threat model; the win detector is independent of grammar membership. This is the in-band/out-of-band split made operational: the behavioral controls fail because the attacker shares their channel, while the grammar is compiled from authenticated identity and applied at the sampler, nowhere the attacker's tokens can reach. Second, the claim is scoped to *action integrity over enumerable-sink tools*; free-text confidentiality through channels that must remain free-text is stated up front as out of scope (Paper-2), consistent with the broader concern that injection through trusted data channels is a structural, not incidental, exposure [7]. Third, the corpus is selected-on-outcome (attacks known effective against the undefended agent, the live-malware analogue), so it is the population of *capable* attacks, not a representative sample of the threat space; bypass rates are conditional on attack potency against the no-defense replay baseline, and the large N tightens intervals without buying generalization.

Two further boundaries scope what the experiment can and cannot underwrite. The "eliminates by construction" claim is carried by the soundness theorem, which quantifies over all logit distributions: it holds against a fully injection-compromised model because it never assumes the logits are benign, giving

$$\text{emitted} \in L(G_s) \subseteq \text{Authorized}(s),$$

and no finite sample could license a class-level elimination claim that this proof does [18]. The experiment instead shows that behavioral controls genuinely leak at scale and that the implementation matches the theory; proof and evidence stay in separate lanes. And the empirical claim itself is a feasibility claim on a single subject system (Tantalus), not a population-level universality claim. The

trusted computing base is correspondingly explicit: the guarantee holds iff $L(G_s) \subseteq \text{Authorized}(s)$ (i.e. iff the compiled allowlist is correct), which is why we recommend the post-parse check as an independent acceptance oracle and executor re-validation backstop, a Condition-`D`-class corrective control deployed as defense-in-depth that complements `C` rather than competing with it.

Finally, external validity remains a boundary we scope carefully rather than overstate. RQ1, the formal H2 contrast, and the blind-`C` utility point estimate are demonstrated on two capable non-Qwen victims (Mistral-119B and Gemma-4-31B); the pre-registration's ≥ 3 distinct pretraining lineage bar is now met (Qwen + Mistral + Gemma), with the caveat that the Gemma result rests on a single 31B anchor (not multi-size coverage; Limitations 8 item 1). On Gemma the behavioral stack leaks far harder than on Mistral: A4 bypass 33.80% [31.73, 35.92] (CP95), roughly 18× Mistral's 1.82%, while GCD again pins to 0.00% [0, 0.18] with 0 emission, and the closed grammar `c_l3_closed` reaches 100.00% [99.41, 100] legitimate-task success at 0 bypass. The C=0 invariant reproduced across two grammar backends (xgrammar for the 1.7B blast and steelman; guidance/llguidance for the Mistral and Gemma anchors [19]) and, as a robustness note, across two additional auxiliary engines not used as primary data sources (`llama.cpp`/GBNF on the 35B-A3B anchor [20]; SGLang/xgrammar on an old-box 1.7B run), supporting engine-independence, not three co-equal anchors. The registered powered 2pp non-inferiority TOST for RQ4 was executed on the Mistral lineage (Mistral ladder, $n = 1{,}456$/cell, 208 `task_id` clusters × 7 rounds, cluster-bootstrap $B = 10{,}000$): the observed difference is -1.85 pp (point estimate inside the margin) but the 95% CI is $[-7.07, +3.43]$ (half-width ≈ 5.3 pp $\gg 2$ pp), so *non-inferiority is not established even when powered*. We therefore report at most a point-estimate ordering (blind-`C` legit-success above Control on the capable model, an 11-percentage-point deficit on the weak one), attribute the deficit to weak-model competence rather than to the grammar, and offer `c_guided` (RQ6: 100.0% legitimate-task success, 0 bypass, 0 deflection on the 1.7B) as the universal resolution. These are scope boundaries, not contradictions, and they bound the claim to exactly what the datasets support.

8 Limitations and Deviations from Pre-Registration

We report these limitations up front, as the pre-registration discipline demands. The three completed datasets establish part of the registered program decisively while leaving named pieces open. We distinguish the two below rather than blur them.

1. **Three pretraining lineages, with a single-anchor caveat on the third.** The registered external-validity bar (pre-registration §12 amendment #1: ≥ 3 distinct pretraining lineages) is now **closed**: Qwen (the 1.7B workhorse, the 35B anchor, and the abliterated 27B), Mistral (119B, Sections 6.2 and 6.3), and Gemma (Gemma-4-31B, Section 6.4). However, the Gemma lineage rests on a single model/checkpoint (`nvidia/Gemma-4-31B-IT-NVFP4`): it is one-anchor evidence for that lineage rather than the multi-size coverage Qwen carries. The cached Nemotron-3-Super-120B was abandoned after all gates passed (100% potency) because its NemotronH hybrid-mamba decode (≈64–72 tok/s aggregate, ≈30× too slow) made the full ladder infeasible within the available compute allocation.

2. **The RQ1 headline rides on the Mistral anchor, not the 1.7B.** On Mistral the full stack `a4` leaks 1.82% [1.43, 2.29] (CI-lower > 0), Control 56.77% clears the registered $\geq 50\%$ potency floor, and the `a1` $\leq$ Control manipulation check holds. The 1.7B workhorse is sub-floor *by design* (Control bypass 18.33%; the superseded ungated `a1` orphan at $\sim 20.3\% \geq$ Control

fails the A1<Control check, as expected of a weak victim). It is the weak-end apparatus that sharpens the C-vs-D contrast, not the headline-leak victim.

3. **Steelman $N = 2{,}000$/cell, not the ADR's $\sim 10{,}000$.** The abliterated-adversary run (Section 6.5) was scaled down because of eager-mode 27B throughput on a single TP=2 instance. `c_sealed` at 0/2000 gives a rule-of-three upper bound of $\sim$0.15% (versus $\sim$0.03% at 10k). The 97.85% $\rightarrow$ 0.00% contrast is unambiguous, but the confidence interval is wider than planned. The 97.85% control exfil rate (CP95 lower bound $\approx$97.1%) is tightly bounded even at $N = 2{,}000$; the wider-interval concern applies only to `c_sealed`.

4. **`c_sealed` is partly true-by-construction.** The forced-denylist trajectory makes 0 exfiltration structural at the sink, so the 0.00% is not a discovered safety property of the grammar. Its empirical content is exactly two things: (a) the paired open cell at 97.85% (1957/2000), where the adversary is real, willing, and able; and (b) the 96.8% (1937/2000) provable forced sensitive-artifact read, the objection-killer against "it never engaged the malicious content." It is reported as a rebuttal exhibit, not main statistics, consistent with ADR 0004. The same structural-ungenerability logic applies in a weaker sense to blind-C's headline result; the distinction is that blind-C is scored by executed effect and the free-text injection surface remained open (see below). The provable read is *engagement evidence*, not a confidentiality result: GCD permits the read and seals only the exfil *action* (confidentiality through free-text channels is Paper-2 scope, Section 7). The per-skill artifact reads do not generalize uniformly: "provably read the *AWS credentials*" (`AKIAT4NTALUS…`) holds for 542; SSH private key 259; cross-user chat 517; ticket business-data 619; the 63-trial remainder are unmapped/multi-message entries that still produced 0 exfiltration.

   The main-statistics blind-C cells face the same structural-ungenerability fact: an out-of-scope sink URL is also ungenerable by construction, so a naive reading of the headline 0-bypass result is likewise partly definitional. We address this exactly as the pre-registration designed for it (Section 6). First, bypass is scored *by effect*: the `tantalus-pipeline::wins` detector fires only on an executed out-of-scope `fetchUrl` reaching an attacker sink, not on any grammar-legal token sequence, so the headline figure is "$0/N$ executed calls reaching such a sink," not "0 grammar-legal tokens." Second, under blind-C the free-text surfaces (`searchFiles.query`, `respondToUser.message`) remain open, so an injection route through an open free-text parameter existed and was scored: the attacker could supply arbitrary content in `searchFiles.query` and `respondToUser.message`, yet was never observed to produce a bypass.

5. **The H3 token-cost conjunct fails on the Mistral anchor (§8).** The registered H3 decision rule is conjunctive: C emission = 0 while D emission > 0, *and* C's attack-trial output-token cost strictly below D's. The emission and validity-dividend conjuncts carry decisively on both victims: C 0 emission versus D 104k–120k (1.7B) and 2,317–3,878 (Mistral). However, the token-cost conjunct **fails** on the capable Mistral anchor: C at 208 output tokens is *not* strictly below `d_r1`'s 203. So the *formal* conjunctive rule is met **only** on the 1.7B (C 119 < D 261–270; the ungated `control` cell at 289 tok is not a D variant). The victim-independent result is the qualitative gate-inertness: C makes the corrective gate inert, with 0 emission, 0 gate-rejection, and 0 stranding. Corrective and detective-posture systems, such as Agent-C [9] and PackMonitor [10], enforce the same output policy post-generation and face the same produce-then-catch cost. The same produce-then-catch pattern characterises the post-generation policy-enforcement class more broadly (CaMeL [11], Progent [12], AgentArmor [15]): the H3 token-conjunct limitation applies to that entire tier, not to any specific implementation, and the strict token ordering is victim-dependent.

The validity dividend (the ~ 41 pp gain in `valid_output` on Mistral, Section 6.2) is not affected by this conjunct failure and is reported separately (Section 7). It is victim-*dependent* in magnitude: on a high-native-validity model such as Gemma it is negligible (Section 6.4).

6. **"0 invalid outputs" is not literally met on vLLM, and `c_guided` is worse than the registered floor.** Per-arm GCD `valid_output` on the 1.7B blast: `c_d_r3` 98.9%, `c_closed_d_r3` 99.9%, `c_guided_d_r3` 97.8%. The residual is free-string truncation: under the Kleene-$*$ free-string rule on vLLM (the `{0,N}` bound is an `xgrammar` compile pathology there), a weak model can ramble the open `respondToUser` message to `max_tokens` and truncate. Pre-registration §12 (angle-shift item 6) registered regular-C 99.35% and C+ 99.87%. We single out `c_guided`'s $\approx$2.2% invalid because it is the **largest** GCD invalidity in this dataset, a fresh disclosed deviation *above* the registered invalid-rate (its terminal reply is the still-open free respond-only string; the fix is a closed canned terminal, as C+ already applies to `respondToUser`). In fairness, `c_d_r3` at 98.9% is *also* fractionally below the registered regular-C 99.35% figure (same Kleene-$*$ free-string cause, same fail-safe behavior), and `c_closed_d_r3` at 99.9% is at the registered C+ figure. All fail safe: a truncated string yields no executable tool call (truncation collapses to "no action," never a wrong action; every such trial is 0 win / 0 emission), and the rates are still far below the $\sim$8–10% parse-fail of non-grammar arms.

7. **Blind-C utility (RQ4) is victim-dependent, and the powered non-inferiority test fails even when run.** The weak 1.7B shows a ≈ 11-pp deficit below Control (blind-C legit 47.0% vs Control 58.3%); the capable Mistral anchor shows blind-C legit 78.85% [72.66, 84.19] *above* Control 73.56%. However, that anchor cell ($n = 208$, `--legit-rounds 1`, CI half-width ~ 6 pp) is under-powered. The pre-registered powered 2-pp non-inferiority TOST clustered by `task_id` (§8/§12, `--legit-rounds 7` $\approx$ 1,456 trials/condition) *was* run, on the Mistral ladder (`mistral119_ladder.db`, Section 6.3): observed C$-$Control $= -1.85$ pp (inside the margin) but cluster-bootstrap ($B = 10{,}000$) 95% CI $[-7.07, +3.43]$, half-width $\approx$5.3 pp $\gg$ the 2-pp margin. Non-inferiority is therefore **not established even when powered**, a confirmed negative rather than an unmeasured gap. What *is* established is twofold. The blind-C deflection deficit does not reproduce on the capable Mistral anchor (Section 6.2), suggesting a victim-capability confound, though the causal partition has not been measured. And `c_guided`/`c_l3_closed` (RQ6, H6) is the universal resolution: bypass 0, deflection 0, legit-success 100.0% (`c_guided_d_r3`, the 1.7B gated cell, 624/624), 100.00% [99.75, 100] (`c_l3_closed`, the capable Mistral ladder), and 100.00% [99.41, 100] (`c_l3_closed`, the Gemma anchor; the Gemma run carries no `c_guided` cell). The registered H4 precondition required a victim with $\geq$ 90% Control legitimate-task success. The Mistral anchor's Control legitimate success was 73.56% [67.01, 79.42], below this threshold. The powered TOST was executed and reported anyway: the confirmed-negative verdict stands, but on a victim that did not meet the trigger condition. The H6 resolution (`c_l3_closed` attains 100% legitimate success) requires per-task pre-enumeration of the authorized action sequence, a grammar-authoring overhead the behavioral arms do not carry. The practical deployment trade-off between blind-C (no per-task authoring, non-inferiority unestablished) and L3-closed (guaranteed utility, per-task specification cost) is not resolved by the current experiments and represents an open engineering question.

8. **Engine/version variance and reproducibility gaps: the primary validity concern (§10).** The five datasets span different vLLM builds: the 1.7B blast on vLLM 0.23.0, the steelman on vLLM 0.17.1 (the only Blackwell build that loaded the abliterated NVFP4 hybrid coherently), and the Mistral anchor, the Mistral ladder, and the Gemma anchor on from-source / nightly v1 builds with the `guidance`/`llguidance` structured-output backend

[18, 19] in place of `xgrammar` (ADR 0005; deviation rows below). All five produce C-emission = 0. The cross-version footguns (AWQ-Marlin hang on `sm120`, GDN-TP load hang on 0.23.0, the LARK/`xgrammar` engine crash, and a corrupt-download episode) are apparatus failures that caused the deviation recorded in §8.1 (the guidance-backend switch); they do not affect the C-emission = 0 result but are recorded as the reason for three of five datasets using a non-registered grammar backend. The reproducibility gap itself is the named primary validity concern of §10: **three of the five finding DBs logged `engine_commit = unknown` at runtime** (the 1.7B blast, the Mistral anchor, and the steelman, whose `ENGINE_COMMIT` env was unset on those victim launches), and were **backfilled out-of-band** (tagged `-oob`) from verified engine image digests in `harness/PROVENANCE.md` (blast NVFP4 rows `vllm-openai-0.23.0`, blast BF16 rows `vllm-therock-0.22.1rc1` for 299,829 early ROCm trials, Mistral `vllm-ms4-fromsrc`, steelman `vllm-openai-0.17.1`). The two 2026-06-24 closeout DBs (the Mistral ladder `mistral119_ladder.db` and the Gemma anchor `gemma31_anchor.db`) *did* stamp `engine_commit` correctly at runtime, so the gap is confined to the three earlier DBs. `model_id` remains a served alias; HF checkpoint revisions are resolved in `PROVENANCE.md` (e.g. Mistral `d57a94c7…`, blast-NVFP4 `2ab3a24a…`). Per-trial MTP-setting and seed logging (required by §6/§10) is unverified for the three backfilled DBs. Because §10 names reproducibility *the* primary validity concern, we treat these as **blocking for paper-grade use**, not tail-end polish; native runtime stamping is required on any further runs beyond the two closeout DBs.

9. **H5 (grammar-compile time): measured out-of-band, registered cold target not met.** The registered overhead envelope is a p99 grammar-compile of < 25 ms (§5). It was measured with an out-of-band FSM-compile probe over the real R2 grammar and both production tokenizers (`harness/h5_grammar_compile_results.md`) [19]: blind-C *cold* compile is p50 38.96 ms / p99 58.64 ms on the Qwen-151k vocab and p50 38.52 ms / p99 61.24 ms (max 65.59 ms) on the Mistral-131k vocab, **above** the 25 ms cold target. Amortized over a run with a single fixed grammar (cache hit), per-trial compile is ≈ 0.003 ms (p99 0.0030 ms, Mistral, $N = 500$). The verdict is measured / qualified: the letter of the 25 ms cold target is not met, but the spirit holds amortized, because the confirmatory runs use one fixed grammar whose FSM is compiled once and reused. (The `c_l2_guided` ≈18.6 KB alternation is much costlier, cold p99 346.6 ms Qwen / 535.5 ms Mistral, but amortizes the same way.)

10. **No formal-proof claim is carried by the empirical samples.** This report and the experiments establish the empirical/implementation tier only: that behavioral controls genuinely leak at scale and that the implementation matches the theory. The class-level "eliminates by construction" claim rests on the soundness argument (Section 4), which quantifies over all logit distributions and so holds against a fully injection-compromised model; the full proof is a companion artifact in preparation. Proof and evidence stay in separate lanes.

8.1 Registered deviations (pre-registration §12)

Dated rows were added to the pre-registration change log once results were in, each recorded with its reason per the cardinal rule below.

- **2026-06-22 – `guidance/llguidance` backend.** Three of five datasets used `guidance` [18, 19] in place of `xgrammar` due to a LARK grammar incompatibility (ADR 0005). Item 8 above and the §6.3 apparatus deviation give the full rationale.

- **2026-06-22 – `c_guided` `valid_output` 97.8% exceeds the registered invalid-rate floor.** The terminal free respond-only string under Kleene $*$ on vLLM truncates on a weak victim; it fails safe (0 win / 0 emission). Full discussion in item 6 above.
- **2026-06-24 – `c_l2_guided` dropped from the Mistral ladder (guidance-backend incompatibility).** Under the `guidance`/`llguidance` backend `c_l2_guided` produced only 30.2% `valid_output` ($\sim 72\%$ empty `raw_json` traces, with `guidance` logging "grammar rejected tokens [0]"): the $\approx$18.6 KB 100-way canned-alternation grammar is a construct `xgrammar` handles (97.8% valid on the 1.7B blast) but `llguidance` does not. `c_l2_guided` was dropped from RUN2 (the Mistral ladder) before data collection began; the ladder instead carries *c*, `c_l3_closed`, and Control. *Reason:* this is an apparatus incompatibility, not a grammar-soundness failure, since the same grammar enforces correctly under `xgrammar`.
- **2026-06-24 – H5 grammar-compile time measured out-of-band.** Cold one-time FSM compile ($\approx$ 59–61 ms p99) exceeds the registered 25 ms p99 cold target; amortized per-trial is $\approx$0.003 ms. Full discussion in item 9 above.

Cardinal rule. Once the confirmatory run began, any deviation from the pre-registration is recorded in §12 with a reason; numbers reported here come from the analysis plan fixed in the pre-registration, not from post-hoc choices. This discipline, evaluating a positive, default-deny, complete-mediation control [1, 2] at scale under a locked protocol, is what licenses reading the constitutive-vs-corrective contrast as a finding rather than a fishing expedition.

9 Related Work

We organize the prior art along the two axes that distinguish the controls in this study: the *security model* (negative, enumerate and detect bad behavior, versus positive, enumerate and admit only authorized behavior under default-deny) and the *enforcement stage* (post-generation, after the model has produced an artifact, versus at-generation, at the decoding boundary). The resulting 2×2, summarized in Table 8, locates every control class and exposes the cell this work occupies. The positive-versus-negative contrast we draw, enumerate-and-admit-the-authorized against enumerate-and-detect-the-bad scored by effect at scale, recapitulates the classical host-security shift from signature antivirus to application allowlisting. However, its intellectual roots lie in a longer lineage of positive-security design that we credit next.

9.1 Foundational lineage

The positive-security stance this work operationalizes descends from four classical lines. Saltzer and Schroeder [1] established *complete mediation* and *fail-safe defaults* (default-deny) as design principles: every access is checked against an explicit policy, and the absence of an explicit permission denies. The *reference-monitor* concept [2] gave this an architectural form, a tamper-proof, always-invoked, verifiable mediator of every access, which the post-generation positive controls in our matrix (Condition `d`) approximate at execution time without the always-invoked guarantee, since the artifact exists before the gate fires. The LangSec programme [3] contributed the *correct-by-construction recognizer*: when the set of admissible inputs is a formal language, a recognizer for that language enforces correctness by making malformed inputs unrepresentable rather than detectable. This is the direct intellectual precursor to grammar-as-policy, where the language *is* the policy. Finally, the object-capability model [4, 5] frames authority as an unforgeable token over a designated action space, held only by principals entitled to it. Most directly, Galloway [6]

gave this lineage an empirical control-evaluation form in host security: a controlled comparison of application allowlisting (a positive, default-deny control) against signature-based malicious-code detection (a negative control). The present study ports that evaluation design into the AI-security domain, substituting the LLM agent's tool-call action space for the host's executable space and the behavioral guardrails for signature detection. Grammar-constrained decoding (GCD) over a per-request authorization grammar G_s sits at the intersection: G_s is the reference monitor relocated to the sampler, compiled per request from authenticated caller identity (an unforgeable capability over the agent's action space) and realized as a correct-by-construction recognizer at the decoding boundary. Among the layers this study adds is a formal soundness core (Section 4): constrained generation can emit only strings in the grammar's language, and the soundness statement $\textit{emitted} \in L(G_s) \subseteq \text{Authorized}(s)$ is quantified over all logit distributions, so it holds against a fully injection-compromised model. That is what licenses a class-level elimination-by-construction claim no finite sample could: the empirical evidence shows that behavioral controls genuinely leak at scale and that the implementation matches the theory.

9.2 The threat: indirect prompt injection and the benchmark landscape

The attacks we study are predominantly *indirect* prompt injections delivered through a trusted data channel rather than the user message. Greshake et al. [7] characterized this class: malicious instructions ride inside content the agent retrieves and trusts (documents, emails, web pages), so the injection occupies the same input channel as any legitimate instruction. This is precisely why the in-band behavioral controls in our matrix, the defensive system prompt (`a1`) and the embedding input classifier on the user message (`a2`), can be overridden: they live in the channel the attacker shares. The agent-security benchmark literature has documented the empirical leakiness of behavioral defenses at scale. The Agent Security Bench [8] evaluated a broad matrix of attack and defense methods across many models over 90,000 test cases, reported high peak attack-success rates, and concluded that prevention-based defenses of the detect-bad class are inadequate. Our confirmatory corpus subjects a single subject system, Tantalus (a locally-deployed AI-security agent), to roughly 27× that benchmark's test-case count. We stress that this buys tighter confidence intervals, not generalization, because the corpus is selected-on-outcome (the population of capable attacks against the undefended agent) and bypass rates are therefore conditional on attack potency. Bypass is scored by effect: the win detector (`tantalus-pipeline::wins`) fires only when the exfiltration marker reaches an out-of-scope sink URL in an executed `fetchUrl` call, not on grammar membership, so a grammar-constrained call to an allowlisted URL scores zero bypass regardless (Section 5).

9.3 Grammar-constrained and structured generation machinery

The mechanism we deploy, constraining the sampler to a formal language, builds directly on the structured-generation line of work. Willard and Louf [18] cast constrained generation as an efficient finite-state-machine guided decoding problem; xgrammar [19] provides a high-throughput pushdown-automaton token-mask backend for context-free grammars; and `llama.cpp` [20] exposes native GBNF grammars at the sampler. Our enforcement rides on this machinery, and the same C-emission-of-zero result reproduces across engines and grammar backends. We treat this cross-engine agreement as an engine-independence robustness corroboration rather than as several co-equal anchors: `vLLM`-xgrammar (the 1.7B blast and the steelman) and `vLLM`-guidance/llguidance (the Mistral anchor) are the confirmatory engines for the reported data, while `llama.cpp`-GBNF (on the 35B anchor) and SGLang-xgrammar (on the 1.7B, prior apparatus) serve as auxiliary corroborations.

The distinction we draw is one of *purpose*, not of plumbing. The dominant deployed use of this machinery is structured-output *shaping*: forcing output to a JSON schema or fixed shape. Shaping is not authorization: a schema constrains the *form* of a tool call, not *which* URL, channel, or file the call may name. In our design the structured-output shape is held constant across every condition (it is the substrate, not a control). The novelty is per-request *authorization* compiled into the grammar from the authenticated caller identity and applied at the decoder, so the action space *is* the policy and a forbidden parameter value is ungenerable rather than merely mis-shaped.

9.4 The 2×2: where the deployed controls sit, and the empty cell

Negative, post-generation. This cell is the industry stack: defensive prompting, input classifiers, output filters, and LLM-judge / reasoning-monitor guardrails. They are probabilistic detectors of bad behavior applied after (or, for prompt-level defenses, in-band with) generation. Output filters in particular are a credential-*signature* denylist, incomplete by construction: business data that carries no credential signature passes through unflagged (our `a3` arm, and the survivor class on the capable anchor in Section 6.2). A recurring real-world hazard in this cell is that several commercial "agent security" products market themselves as preventative or allowlist-based while being, mechanically, detectors. This is the motivating gap this study addresses. We name these product classes in prose only; no citation is asserted for them.

Positive, post-generation. The academic frontier has moved security into this cell: deterministic, execution-time reference monitors and policy compilers that admit only authorized actions but enforce the policy *after* the model has generated the tool call. The action is produced in full, then a separate gate inspects and rejects it. Several recent systems realize this class. CaMeL [11] and Progent [12] enforce authorization post-generation via dual-LLM and programmable privilege-control architectures; MiniScope [13] applies least-privilege permission scoping at tool-call time, enforcing post-generation; FIDES [14] uses information-flow control (applied to the generated artifact's provenance graph post-production, placing it in this cell rather than at-generation; it extends into Paper-2 scope for free-text channels); AgentArmor [15] performs program analysis over agent runtime traces; OAP [16] provides deterministic *pre-execution* authorization (pre-execution, not pre-generation, so the artifact still exists before the gate); and ARM [17] identifies a risk that reference monitors in this class can leak information through the denial feedback itself, a surface that emission = 0 structurally forecloses. Our Condition `d` is a faithful instance of exactly this class: it applies the *same* allowlist that G_s compiles (C and D call the same `tantalus_grammar::allowlist_verdict` routine, so $D \equiv C$ by construction), but correctively. This is the right comparator, and it is effective at blocking the effect; we do not claim C is more secure than a perfectly wired allowlist on the blocked effect. The two controls enforce the same positive policy over the same language $L(G_s)$ and differ in enforcement completeness, not in what they permit. The corrective allowlist re-introduces, one layer down, the failure modes positive security exists to escape: the dangerous artifact is produced (a cost and an output-channel / forensics surface, the emission signature), a coverage obligation across every output path and consumer, and a time-of-check/time-of-use window [1]. GCD has none of these because there is nothing to catch. Accordingly we deploy a post-parse check of this class as a defense-in-depth backstop *complementing* C, not as a control C dominates.

Positive, at-generation (this work). The remaining cell, positive security enforced at the decoding boundary, was the least occupied prior to this work. The nearest decode-time neighbours are Agent-C [9], which enforces temporal constraints at the sampling boundary with a model-agnostic soundness claim, and PackMonitor [10], which eliminates package-hallucination via a decode-time

automaton monitor that is the mechanism twin of ours (a DFA driving a logit mask). Both demonstrate the viability of decode-time enforcement but differ in scope: Agent-C constrains action *ordering* (and checks temporal formulae by SMT-and-resample rather than masking the forbidden token to zero), and PackMonitor constrains package-name *enumeration* for hallucination-correctness, whereas this work compiles per-request *authorization* policy from authenticated caller identity (the action space *is* the policy) and masks every out-of-scope token to zero. Agent-C's resample-until-valid approach leaves each individual draw able to produce a constraint-violating token; GCD's mask-to-zero makes every forbidden token ungenerable at any draw, which is the locus of the no-artifact guarantee. We therefore claim to be the first to *build and measure* generation-time authorization at scale across multiple model families with formal soundness, not the first to conceive decode-time enforcement. GCD at the sampler moves enforcement one stage earlier than the execution-time monitor: the grammar mask is applied before sampling, driving every grammar-invalid token's probability to zero, so for enumerable-sink parameters the out-of-scope action is ungenerable by any sampler and never exists as an artifact on any surface (output buffer, logs, telemetry, stream, KV cache). Free-text fields (`searchFiles.query`, `respondToUser.message`) remain open-vocabulary; free-text confidentiality leakage is out of scope for the headline security claim (Section 8). The locus is the sampler, not the model: the transformer still computes a full logit distribution and under injection may assign an arbitrarily high logit to a forbidden token, but that intent never becomes a generated token. Because the grammar is compiled from authenticated identity and applied at the decoder, it is out-of-band, nowhere the attacker's injected tokens can reach, which is the precise reason the in-band behavioral controls fail and this one holds. This cell is where enforcement is provable and leaves no emission. With xgrammar (which removes the per-token CPU-mask cliff) C is at-par with free decoding in the agentic loop, and the per-token masking cost is reported honestly against the avoided regeneration. A structural corollary is a model-agnostic *validity* guarantee: every emitted token is in $L(G_s)$ by construction, so valid-output rates are decoupled from model capability. A 1.7B victim under GCD reaches 98.9% valid outputs (Section 6.1, `c_d_r3`) versus 90% under control (89.4–93.9% across behavioral conditions), independent of the model's fluency. However, the validity dividend is victim-dependent: it is largest where native free generation is noisy (≈59% on Mistral, 89–94% on the weak 1.7B) and negligible where the model already produces well-formed tool calls natively (Gemma, Section 6.4). In both regimes GCD validity is model-agnostic by construction. This is a reliability contribution separate from the bypass result. The enforcement locus is also distinctive: the mask is applied at every emitted token rather than after a finished call, so it acts on the free-text and streaming surfaces an execution-time monitor inspects only after the fact, a statement about *where* enforcement happens, scoped to action integrity over enumerable-sink tools.

9.5 Abliteration and the adversarial upper bound

Behavioral safety can be surgically removed from a model's weights. Arditi et al. [21] showed that refusal behavior in open-weight models is mediated by a single linear direction in the residual stream that can be ablated, yielding a model that cannot refuse. This is the natural steelman for our thesis: abliteration defeats every weight- and prompt-resident behavioral defense (the negative/post-generation cell) by construction, because there is no refusal left to elicit. It cannot, however, touch a constraint that lives in the sampler. Section 6.5 reports the resulting contrast on a fully abliterated model under a top-authority malicious prompt: 97.85% exfiltration with the sink open versus 0.00% under a sampler-sealed grammar, with the grammar as the sole independent variable. We present this as a rebuttal exhibit rather than a primary statistic, and we are explicit (Section 8) that the sealed cell is true-by-construction. Its empirical content is the paired open cell (the adversary is

Table 8: The design space as a 2 × 2: security model × enforcement stage. Each cell names the deployed control class and its nearest published neighbours; Condition labels in `typewriter`. The positive/at-generation cell is the contribution of this work.

	Negative (detect-bad)	**Positive** (allowlist, default-deny)
Post-generation	Behavioral / LLM-judge guardrails: defensive prompt (`a1`), input classifier (`a2`), credential output filter (`a3`), full stack (`a4`) [7, 8]; commercial "agent security" marketed as preventative but mechanically a probabilistic detector.	Deterministic reference monitors / policy compilers / execution-time allowlists: CaMeL [11], Progent [12], MiniScope [13], FIDES [14], AgentArmor [15], OAP [16], ARM [17] (Condition `d` class). Same policy as C; the artifact is produced then caught, so emission, coverage, and TOCTOU surfaces remain.
At generation	—[†]	**GCD at the sampler (`c/c_guided`), this work.** Nearest neighbours: AgentC [9] (temporal ordering, SMT-resample) and PackMonitor [10] (package-name DFA), distinguished by per-request authorization scope and mask-to-zero. Provable (Section 4), no emission, at-par with free decoding in the agentic loop (vLLM/xgrammar); the mask applies per-token, not after a finished call (action-integrity scope: enumerable-sink parameters).

[†]No published system is known to enforce a negative/detect-bad policy at the decoding boundary; the cell is structurally empty because detect-bad and at-generation are jointly incoherent (detection requires comparing against an artifact, which does not yet exist at the decoding boundary).

real, 97.85% exfiltration) and the 1,937/2,000 (96.8%) provably-forced sensitive reads across all mapped skills (the engagement-evidence objection-killer), not a claim that the grammar discovered safety.

10 Conclusion and Future Work

We establish constitutive enforcement at the decoding boundary, positive security all the way down to the generative act [1, 2], as a structural control for LLM agent tool calls within the Tantalus agent system (§5), at scale, with a soundness core and an explicit cost axis. The central distinction is *constitutive vs. corrective* enforcement. GCD (Condition `c`) and a post-parse allowlist (Condition `d`) enforce the *same* positive policy over the same language $L(G_s)$: both compute it through the single recognizer `tantalus_grammar::allowlist_verdict`, so the policy is correct-by-construction [3] and G_s, compiled from the authenticated principal's identity, is an unforgeable per-request capability [4, 5]. They differ only in where that policy sits relative to generation. Under `c` the policy *is* the output space, so an out-of-scope action is ungenerable: it never exists as an artifact in the output buffer, logs, telemetry, or stream.[4] Under `d`, however, an unbounded default-allow generator *produces* the malicious action in full and a separate gate then rejects it, re-introducing

[4]Architecturally the token cannot enter the KV cache either, since it was never sampled; this is a design consequence, not a separately measured property of this study.

one layer down the very failure modes positive security exists to escape: the artifact is produced (cost plus an output-channel forensics surface), the gate carries a complete-mediation obligation on every output path [1], and a TOCTOU window opens. GCD's positivity is enforcement-deep, positive security all the way down to the generative act, while the allowlist's positivity is policy-deep only; the two are policy-equivalent, with enforcement-completeness differing. Post-generation policy enforcement (Condition `d` and systems such as CaMeL [11], Progent [12], OAP [16], and AgentArmor [15]) is policy-deep. The decode-time neighbours Agent-C [9] and PackMonitor [10] share the enforcement-deep lane but enforce temporal and hallucination constraints respectively, not authorization over enumerable sinks; §9 gives the full distinctions.

Five datasets across three pretraining lineages substantiate this. On a deliberately weak workhorse (Qwen3-1.7B-NVFP4), a generator×gate factorial at ∼282,000 attack trials per cell reproduces the structural result decisively: the GCD arms emit *zero* out-of-scope artifacts (scored by effect, where the attacker URL plus marker must traverse the free-text surfaces that remain open under blind-`c`, `searchFiles.query` and `respondToUser.message`, via `tantalus-pipeline::wins`) and render the gate inert (`c_d_r3` emission 0, `gate_rej` 0), while behavioral generators behind the identical gate produce 104,615–120,140 out-of-scope calls each at roughly twice the output tokens. On a capable non-Qwen anchor (Mistral-Small-4-119B-NVFP4), the behavioral arms run *ungated* and deliver what the weak victim could not. RQ1 holds: the full stack `a4` still leaks 1.82% [1.43, 2.29] (CI-lower > 0) and `a1` (56.50%) $\leq$ Control (56.77%). The registered H2 contrast holds: `c` sits at 0.00% [0, 0.09], strictly below every behavioral point estimate. GCD also decouples output validity from model capability. On the capable Mistral-119B anchor, behavioral arms produced parseable tool calls only $\approx$ 59% of the time while `c` achieved 99.8% (Tab. 4), a validity dividend that is independent of bypass rate and holds regardless of attack potency. Its magnitude, however, is victim-dependent: $\approx$ 41 pp on the Mistral anchor (behavioral $\approx$ 59.2% vs. `c` 99.8%) but near-zero on Gemma, whose native free-generation is already 97.9–100% valid (Tab. 5). A separate abliterated-adversary steelman, a maliciously prompted Qwen3.5-27B that is demonstrably willing and able (97.85% bypass unconstrained [21]), is driven to 0.00% by the grammar alone (97.85% $\rightarrow$ 0%, grammar the sole IV), while provably ingesting its skill's sensitive artifact in 96.8% of trials.

This last result is a rebuttal exhibit, not main statistics. It is engagement evidence (the model engaged the malicious content), not a confidentiality result, and `c_sealed`'s 0% rests on both the structural guarantee (attacker sink ungenerable) and the empirical forced-trajectory trace (96.8% of trials provably ingested the sensitive artifact yet emitted 0/2,000 attacker `fetchUrl` calls).

The honest envelope is narrower than the headline. External validity rests on three pretraining lineages (Qwen, Mistral, Gemma), and the registered $\geq$3-lineage bar is met on a point-estimate basis: three pretraining lineages replicate H1 and H2 (Gemma anchor: control 98.85%, `c` 0.00% [0, 0.18], `a4` 33.80% [31.73, 35.92] CI-lower > 0). However, the Gemma anchor is a single-anchor result at $n = 2{,}000$ attack trials/cell (vs. 4,000 on Mistral and 282,000 on the 1.7B) on a single vLLM engine, so full replication-weight closure awaits a fourth lineage (§8). The RQ1 leak and the formal H2 contrast are carried by the capable Mistral and Gemma anchors, not by the 1.7B (which is sub-floor by design at 18.33% Control bypass and is the weak-end apparatus, not the headline victim). The registered powered 2pp non-inferiority TOST clustered by `task_id` was run on the Mistral anchor (`mistral119_ladder.db`, 1,456 legit trials/cond, `--legit-rounds 7`) and confirmed not established: observed `c`−Control $= -1.85$ pp (inside the 2 pp margin) but cluster-bootstrap 95% CI [−7.07, +3.43] (half-width $\approx$ 5.3 pp $\gg$ 2 pp), so the CI does not clear the non-inferiority margin even with the powered design (H4 not established). "0 invalid outputs" is likewise not literally met on vLLM (regular-`c` 98.9% valid, `c_closed_d_r3` 99.9%, `c_guided_d_r3` 97.8%, all from free-string truncation under Kleene `*` that fails safe). The C-guided (L3-closed)

superiority result (H6), `c_l3_closed` bypass 0, deflection 0, legit-success 100.0%, holds across all three lineages: Qwen-1.7B (624/624), Mistral-119B ladder CP95 [99.75, 100], and Gemma-31B CP95 [99.41, 100]. Grammar-compile time (H5) is measured out-of-band but not met at cold-compile: blind-`c` p99 $\approx$ 59–61 ms (Qwen/Mistral tokenizers) against the registered 25 ms target, though the spirit holds amortized ($\approx$0.003 ms per trial at cache hit). The single subject system (Tantalus) makes this a feasibility claim, not a population-level one. The corpus is selected-on-outcome (the live-malware analogue), so bypass rates are conditional on attack potency, and large N tightens confidence intervals [22] without buying generalization. The class-level "eliminates" claim is carried by the soundness theorem, which quantifies over all logit distributions and so holds against a fully injection-compromised model, not by any sample; proof and evidence stay in separate lanes.

Future work. The open items follow directly from this envelope:

- **A fourth pretraining lineage** (Llama-family or a dense Gemma variant) to replicate H1/H2/H3/H6 beyond the current three (§8). `Nemotron-3-Super-120B` was disqualified due to hybrid-mamba decode throughput ($\approx$64–72 tok/s aggregate, $\approx$30$\times$ too slow for a full ladder) despite passing all potency gates.
- **Capable-anchor C+ (L2-guided) closed-response cell.** The Mistral ladder delivered H6 on a capable model via `c_l3_closed` (§6.3); what remains is the capable-anchor `c_plus` (L2-guided closed-response) cell, which would extend the closed-response validity result to a capable non-Qwen model.
- **Tightening the H4 non-inferiority CI.** The powered 2pp TOST clustered by `task_id` (`--legit-rounds 7`, 1,456 trials/cond) has been run and fails (§6.3; Table 7), so the open item is to explain why the cluster-level precision is insufficient and what sample size or cluster structure (more `task_id` clusters, more rounds per cluster) would shrink the half-width below the margin.
- **Closing the cold-compile H5 deviation.** Cold FSM compile (xgrammar [19] / guidance) is measured at p99 $\approx$59–61 ms against the registered 25 ms target; closure on a production-scale tokenizer or a tighter grammar is outstanding (the amortized per-trial cost $\approx$0.003 ms already meets the spirit at cache hit).
- **The full mechanized soundness proof** as a companion artifact, promoting the proof sketch's caveats to discharged hypotheses and making the for-all-logit lemma and the tokenizer-lifting argument first-class.
- **Paper-2: free-text confidentiality**, covering enumerable-response `c_plus` where the response space is enumerable, plus layered DLP for genuinely open-vocabulary channels (`searchFiles.query`, composed `respondToUser.message`) that lie outside this paper's action-integrity scope.

The thesis, restated, is structural and narrow. Enforcement at the decoding boundary, the grammar mask applied at the sampler before any token is drawn, is positive security all the way down to the generative act [1, 2]. The transformer's logits encode intent and may be arbitrarily compromised; the generated tokens are constrained regardless, so the unauthorized action is not caught but ungenerable, with the decoded tool call $\in L(G_s) \subseteq \text{Authorized}(s)$.

References

[1] J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[2] J. P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, U.S. Air Force Electronic Systems Division, 1972. (Origin of the reference-monitor concept.)

[3] L. Sassaman, M. L. Patterson, S. Bratus, and M. E. Locasto. Security applications of formal language theory. *IEEE Systems Journal*, 7(3):489–500, 2013. (The LANGSEC programme: correct-by-construction recognizers.)

[4] J. B. Dennis and E. C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

[5] M. S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006. (Object-capability security.)

[6] G. R. Galloway. *Application Whitelisting as a Malicious Code Protection Control*. Doctoral dissertation (D.Sc. in Cybersecurity), Capitol Technology University, October 2020. (The direct host-security precursor: a controlled evaluation of application allowlisting as a positive, default-deny control against signature-based malicious-code detection — the comparison this work ports into the LLM-agent domain.)

[7] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[8] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang. Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.02644.

[9] A. Kamath, S. Zhang, C. Xu, S. Ugare, G. Singh, and S. Misailovic. Enforcing temporal constraints for LLM agents. arXiv:2512.23738, 2025. `https://arxiv.org/abs/2512.23738`.

[10] X. Liu, Y. Liu, Y. Zhang, J. Li, and S.-M. Hu. PackMonitor: Enabling zero package hallucinations through decoding-time monitoring. arXiv:2602.20717, 2026. `https://arxiv.org/abs/2602.20717`.

[11] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr. Defeating prompt injections by design (CaMeL). arXiv:2503.18813, 2025. `https://arxiv.org/abs/2503.18813`.

[12] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song. Progent: Securing AI agents with privilege control. arXiv:2504.11703, 2025. `https://arxiv.org/abs/2504.11703`.

[13] J. Zhu, K. Tseng, G. Vernik, X. Huang, S. G. Patil, V. Fang, and R. A. Popa. MiniScope: A least privilege framework for authorizing tool calling agents. arXiv:2512.11147, 2025. `https://arxiv.org/abs/2512.11147`.

[14] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin. Securing AI agents with information-flow control (Fides). arXiv:2505.23643, 2025. https://arxiv.org/abs/2505.23643.

[15] P. Wang, Y. Liu, Y. Lu, Y. Cai, H. Chen, Q. Yang, J. Zhang, J. Hong, and Y. Wu. AgentArmor: Enforcing program analysis on agent runtime trace to defend against prompt injection. arXiv:2508.01249, 2025. https://arxiv.org/abs/2508.01249.

[16] U. Uchibeke. Before the tool call: Deterministic pre-action authorization for autonomous AI agents. arXiv:2603.20953, 2026. https://arxiv.org/abs/2603.20953.

[17] M. H. Chinaei. Causality laundering: Denial-feedback leakage in tool-calling LLM agents. arXiv:2604.04035, 2026. https://arxiv.org/abs/2604.04035.

[18] B. T. Willard and R. Louf. Efficient guided generation for large language models. arXiv:2307.09702, 2023.

[19] Y. Dong, C. H. Yin, et al. XGrammar: Flexible and efficient structured generation engine for large language models. arXiv:2411.15100, 2024.

[20] G. Gerganov and contributors. llama.cpp: LLM inference in C/C++ with GBNF grammar-constrained sampling. Software. https://github.com/ggml-org/llama.cpp.

[21] A. Arditi, O. Obeso, A. Syed, D. Paleka, N. Panickssery, W. Gurnee, and N. Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[22] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.

[23] J. N. Amaral et al. About computing science research methodology. Methodology primer.